

UPDATE

MCSL 382 Release

PluginHive MCSL · Shopify / WooCommerce / BigCommerce / Magento / PrestaShop

| Story / card | Platform | Carrier scope | Toggle / prerequisite signal | Trello card |

Generated June 2026 · PluginHive QA Team

Included Story Cards

Story / card	Platform	Carrier scope	Toggle / prerequisite signal	Trello card
ZI-564	Shopify, Magento, PrestaShop	UPS	None detected	LAMSPjle
ZI-565	Shopify	UPS	None detected	Ru0KCBpe
ZI-566	Shopify	TForce Freight	None detected	QjX9VGza
ZI-567	Shopify	DHL Express	None detected	Tq9glgfv
ZI-568	Shopify	FedEx, DHL	None detected	gao3uv58
ZI-570	Shopify	FedEx, UPS	None detected	y7WZOyyq
ZI-571	WooCommerce	Carrier-neutral	None detected	4C0YrEax
ZI-574	Shopify	Amazon	gated	ku0XyzRY
ZI-575	Shopify	UPS, Canada Post	all.selective.manifest.enabled, ON scenarios, gated	edxRPQIk
ZI-576	Shopify	FedEx, UPS, DHL	fedex.rest.saturday.delivery.rates.enabled, Rate Summary	NpGp3YGF
ZI-577	Shopify	FedEx, UPS	None detected	orlX1VP
ZI-578	Shopify	UPS, DHL	None detected	wKY8tldq
ZI-579	Shopify	FedEx, UPS	accountuuid.show.and.edit.signature.on.quick.summary.details.section.enabled, 673a1996-1a9d-4cc1-9c59-527b3f9c5f20.show.and.edit.signature.on.quick.summary.details.section.enabled	vUBfAgZv
ZI-580	Magento	PostNL	None detected	jrUM4aUB

Story / card	Platform	Carrier scope	Toggle / prerequisite signal	Trello card
Migrate FedEx REST DG, Alcohol, and Battery shipment support from C2 to C39 and Implemented ORM_D dg shipment	Shopify	FedEx, UPS	None detected	U02MSYJq
Iteration 2: FedEx REST DG Close Manifest – Show Confirmation Modal Indicating Manifest Will Be Generated for All FedEx Ground Closed Label Orders	Shopify	FedEx	gated., only show the manifest-confirmation modal when DG is involved and the carrier service is FedEx Ground.	cPWaVpoL

How Support Should Use This Package

- › Use each card section as the support/demo guide for that feature.
- › Confirm customer/test platform, live store, carrier account, order/product, and toggle state before promising behavior.
- › Capture the escalation packet listed in the relevant card section if behavior does not match.

ZI-564 - From SL: ZI-564 — Deleted Shopify products persist in StorePep product list [#277997]

Support Guide: ZI-564 — Deleted Shopify Products Persist in MCSL Product List

From SL: ZI-564 — Deleted Shopify products persist in StorePep product list [#277997]

Feature Summary

Prior to this fix, products deleted from Shopify Admin were not removed from the MCSL App Products list. The app had no mechanism to receive or act on Shopify's `products/delete` webhook, leaving stale product rows visible in hamburger menu > Products. This caused confusion during label generation and product-based shipping rule setup. The fix introduces a webhook listener for `products/delete` events, gated by a per-account toggle and a store creation date threshold, so that deletions in Shopify now propagate cleanly to the MCSL product list in near real-time. This affects all stores using the MCSL app on Shopify where product sync is active; UPS is the carrier referenced in the card context, but the fix is product-sync-layer behaviour, not carrier-specific.

Toggles & Prerequisites

Toggle / config note	What support should confirm
<code>{accountUUID}.product.delete.webhooks.enabled: true</code>	Confirm this flag is enabled for the merchant's account UUID before troubleshooting any delete-sync issue. If absent or false, the webhook handler will not fire.

Toggle / config note	What support should confirm
<code>product.delete.webhooks.enabled.after.store.createdAt.greater.than</code>	Confirm the store's creation date meets or exceeds the configured threshold date. Stores created before the cutoff date may not have the webhook registered. Confirm exact threshold value from live card/backend config.
Shopify products/delete webhook registration	Confirm the webhook is registered in the merchant's Shopify store. Can be cross-checked via Shopify Admin > Settings > Notifications > Webhooks or via MCSL request log for incoming webhook events.
MCSL release prerequisite	This fix ships in SL MCSL 382 (Iteration backlog). Confirm the merchant's app version is on or after this release before investigating.
Platform	Shopify only for this webhook mechanism. Magento and PrestaShop are listed as test platforms — confirm from live store/card context whether product-delete sync behaviour applies on those platforms via a different mechanism.
UPS carrier account	UPS is referenced in the card context. Confirm from live store/card context whether the merchant's issue is isolated to UPS-linked products or is platform-wide.

Step-by-Step Support Walkthrough

1. Confirm the merchant's MCSL app version and account toggle.

- › Ask the merchant or check internally that the store is running SL MCSL 382 or later.
- › Confirm `{accountUUID}.product.delete.webhooks.enabled` is set to `true` for this account.
- › Confirm the store's `createdAt` date satisfies the `product.delete.webhooks.enabled.after.store.createdAt.greater.than threshold`.
- › If either condition is not met, the webhook handler is inactive — this is the first thing to rule out before any further investigation.

2. Reproduce or verify the delete event on the Shopify side.

- › In **Shopify Admin > Products**, confirm the product in question has been fully deleted (not archived). Archived products do not trigger the `products/delete` webhook and will not be removed from MCSL (TC-13 confirmed this distinction).
- › Note the Shopify product ID of the deleted product for log correlation.

3. Check the MCSL App Products list for the stale row.

- › In the MCSL app, navigate to **hamburger menu > Products**.
- › Search or scroll for the deleted product by title or Shopify product ID.
- › If the row is still present after a reasonable webhook propagation window (allow up to 60 seconds), proceed to log verification.
- › If the row is absent, the fix is working as expected — confirm with the merchant that a page refresh is required (TC-10: the row disappears on refresh, not in real-time without a reload).

4. Inspect the MCSL request log for the incoming `products/delete` webhook.

- › Navigate to **hamburger menu > Settings > Request Log** (or the equivalent log path in the MCSL app).
- › Filter or search for `products/delete` events matching the Shopify product ID noted in step 2.
- › Confirm the webhook was received and returned a 2xx response.

- › - Request/log fields to verify: webhook topic = products/delete, Shopify product ID, HTTP response status (expect 2xx), timestamp within propagation window
- › If no products/delete entry appears in the log, the webhook is not being delivered — escalate with the Shopify webhook registration status and the account toggle values.

5. Verify variant-level delete behaviour if the merchant reports partial removal.

- › If the merchant deleted specific variants (not the whole product), navigate to **hamburger menu > Products** and confirm only the deleted variants are removed while remaining variants are still listed (TC-2, TC-8).
- › If the merchant deleted the last remaining variant on a single-variant product, confirm the parent product row is also cleared (TC-7 in QA notes).
- › If the merchant deleted an entire colour option group (all variants of a given option), note this is a known outstanding case (whole-color-option removal — backend bulk-event throughput gap, separate dev follow-up). Do not promise resolution on this specific sub-case without confirming fix status from the development team.

6. Verify label generation is not broken by the deleted product.

- › Navigate to the **ORDERS** tab, open the relevant order, and click **Prepare Shipment / Generate Label**.
- › Confirm that orders placed before the product was deleted still generate labels successfully. The order snapshot (title, weight at order time) should be used — label generation should not fail with a "product not found" error (TC-7 in QA notes).
- › - Request/log fields to verify: order line item product ID, weight/dimensions used in carrier request, carrier response status (expect successful label)

7. Verify the recreate/zombie regression scenario if the merchant reports a duplicate row.

- › If the merchant deleted a product and recreated one with the same title, navigate to **hamburger menu > Products** and confirm exactly one row exists for that title.
- › Confirm the Shopify product ID shown in MCSL matches the newly created product's ID, not the previously deleted one (TC-4).
- › - Request nodes to verify: products/create webhook payload — confirm new product ID does not match the previously deleted product ID

8. Validate on Magento or PrestaShop if the merchant's store is on those platforms.

- › The card lists Magento and PrestaShop as test platforms. If the merchant is on **Magento Admin > PluginHive Multi Carrier Shipping Label extension** or **PrestaShop Admin > PluginHive Multi Carrier Shipping Label module**, confirm from live store/card context whether the product-delete sync mechanism applies on those platforms, as the Shopify webhook approach is Shopify-specific.
- › For Magento: check **Magento Orders/Products** and shipping logs for any stale product entries.
- › For PrestaShop: check **PrestaShop Orders/Products** and shipping logs for equivalent behaviour.
- › Escalate to the development team if the behaviour on Magento or PrestaShop is unclear from the card context.

Expected Behaviour

What support should observe	How to confirm
Deleted Shopify product row is removed from MCSL App Products after a page refresh	Navigate to hamburger menu > Products; search by product title or ID — row must be absent
All variants of a deleted multi-variant product are removed	Filter MCSL Products list by product title; no variant rows should remain
Partial variant delete removes only the deleted variants, leaving remaining variants intact	Filter MCSL Products list; only deleted variant rows are gone
Bulk-deleted products (up to 5 tested) are all cleared from MCSL	MCSL Products list count decreases by the exact number of bulk-deleted products
Recreated product with the same title appears as exactly one row with the new Shopify product ID	MCSL Products list shows one row; product ID matches the new Shopify product, not the old deleted one
Duplicate products/delete webhook (Shopify retry) is handled idempotently — no error, no duplicate	Request log shows 2xx response for the replayed webhook; MCSL Products list unchanged
Archived products are NOT removed from MCSL (archive ≠ delete)	Archive a product in Shopify; confirm it remains in MCSL Products list
Product edit/rename does not trigger removal from MCSL	Edit product title in Shopify; MCSL Products list still shows the product with updated title
Orders placed before product deletion still generate labels successfully	Open order in ORDERS tab > Prepare Shipment / Generate Label; label generates without "product not found" error
products/delete webhook received and acknowledged with 2xx in MCSL request log	hamburger menu > Settings > Request Log; filter by topic products/delete and confirm 2xx status
Whole-Color-option removal (all variants of a colour option deleted in one batch) may not fully clear — known open item	Confirm from dev team before committing to resolution; do not mark as fixed for this sub-case

Merchant-Safe Explanation

When you delete a product in Shopify, the PluginHive Multi Carrier Shipping Label app now automatically removes that product from its internal product list. Previously, deleted products would remain visible in the app even after being removed from your Shopify store. With this update, once you delete a product in Shopify Admin, it will no longer appear in the app's Products section after a page refresh — keeping your product list accurate and preventing any confusion during label generation or shipping rule setup. Please note that archiving a product in Shopify is different from deleting it; archived products will remain in the app's list until they are fully deleted.

Common Questions & Troubles

ZI-565 - From SL: ZI-565 — Product name or variant changes create duplicate entries in StorePep [#277997]

Support Guide: ZI-565 — Product & Variant Rename Deduplication in MCSL

From SL: ZI-565 — Product name or variant changes create duplicate entries in StorePep [#277997]

Feature Summary

Previously, when a merchant renamed a product or variant in Shopify Admin, the MCSL app (StorePep) created a new row for the updated name instead of updating the existing one — leaving stale duplicate entries in the MCSL Products list. This fix changes the upsert logic so that product and variant records are matched by their Shopify product ID and variant ID (not by title), ensuring renames, variant additions, and variant deletions all update the existing MCSL row in place. Duplicate rows in the Products list directly affect rate calculation, packaging rules, and label generation, so this fix prevents wrong-product selection and catalog bloat. This change is part of MCSL SL 382 and applies to the Shopify platform. No specific carrier is gated, though UPS is the noted carrier context for this card.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggle detected	Fix is always-on after deployment; no merchant-side toggle required
MCSL SL 382 release	Confirm the store is on the SL 382 build or later before troubleshooting
Shopify platform only	This card targets Shopify webhook handling; confirm the merchant's store is on Shopify
PH Multi Carrier Shipping Label app installed	Confirm app is active in Shopify Admin > Apps
Shopify products/update webhook	Confirm webhook is registered and delivering to MCSL (check request log for recent products/update events)
UPS carrier account	Noted as the carrier context; confirm UPS is configured under hamburger menu > Carriers if rate/label issues are reported alongside duplicate products
No carrier account prerequisite for the dedup fix itself	The rename dedup logic is not carrier-gated; UPS is context only

Step-by-Step Support Walkthrough

1. Confirm the app version and release.

Ask the merchant or check internally that the store is running MCSL SL 382 or later. If the store is on an earlier build, the fix is not present — escalate for a release update before continuing.

2. Reproduce or confirm the reported duplicate scenario.

Ask the merchant which product or variant was renamed and when. Establish the Shopify product ID (visible in the Shopify Admin product URL: `admin.shopify.com/store//products/`). Note this ID — it is the key used for dedup verification.

3. Check the MCSL Products list for duplicates.

In the MCSL app: **hamburger menu > Products**.

- › Search or scroll for the product by its current (renamed) title.
- › Check whether the old title still appears as a separate row.
- › Count total rows for that product. After the fix, exactly one row should exist, keyed to the Shopify product ID.
- › If two rows exist (old title + new title), the fix has not taken effect or the store is on a pre-382 build.

4. Verify the `products/update` webhook was received and processed.

In the MCSL app: **hamburger menu > Settings > Request Log** (or the equivalent request log path in the app).

Navigate to the log entries around the time of the rename.

- › - Request/log fields to verify: `webhook topic = products/update`, `Shopify product id` matching the renamed product, `HTTP response status = 2xx`
- › Confirm the log shows a successful (2xx) response for the `products/update` event.
- › If the log shows no entry for that product update, the webhook may not be registered or may have failed delivery — check **Shopify Admin > Settings > Notifications > Webhooks** for delivery status.

5. Verify variant-level dedup (if the report involves a variant rename or deletion).

In the MCSL app: **hamburger menu > Products**.

- › Expand the affected parent product row to view its variants.
- › For a **variant rename**: confirm only one variant row exists with the new title; the old variant title should not appear as a separate entry.
- › For a **variant deletion**: confirm the deleted variant no longer appears; the remaining variants and the parent product row should be unchanged.
- › For a **variant addition**: confirm the new variant appears under the same parent product row with no duplicate parent created.
- › - Request/log fields to verify: `webhook topic = products/update`, `variant id` matching the renamed/deleted/added variant, `HTTP response status = 2xx`

6. Verify idempotency (duplicate webhook replay).

If the merchant or Shopify has resent the `products/update` webhook (Shopify can retry on failure), check the request log for multiple entries for the same product ID.

- › - Request/log fields to verify: multiple `products/update` entries for the same Shopify product id — confirm each returns 2xx and the Products list still shows exactly one row for that product
- › If multiple rows appear after a webhook retry, this is a regression — escalate with log evidence.

7. Cross-check Shopify source data.

In **Shopify Admin > Products**, open the affected product and confirm:

- › The current product title matches what MCSL now shows.
- › The product ID in the URL matches the ID of the single MCSL row.
- › Variant titles in Shopify Admin match the variant entries shown in **hamburger menu > Products** in MCSL.

8. Verify downstream flows are unaffected.

If the merchant reports rate or label issues alongside the duplicate, open a test order containing the renamed product in the **ORDERS tab > open order > Prepare Shipment / Generate Label**.

- › Confirm the correct (non-duplicate) product is being used for rate calculation and label generation.
- › If a stale duplicate row was previously selected in a packaging or automation rule, ask the merchant to review **hamburger menu > Settings > Packaging** and **hamburger menu > Settings > Shipping Rates / Automation** to ensure rules reference the current product row, not a now-removed duplicate.

Expected Behaviour

What support should observe	How to confirm
Renaming a product in Shopify updates the existing MCSL row in place — no new row created	hamburger menu > Products: one row with new title, old title absent, same Shopify product ID
Total product count in MCSL is unchanged after a rename	hamburger menu > Products: count before and after rename is identical
Adding a variant appends to the existing parent product row — no duplicate parent	hamburger menu > Products > expand parent: both old and new variants visible under one parent row
Renaming a variant updates the variant row in place — old variant title absent	hamburger menu > Products > expand parent: one variant row with new title, old title gone
Deleting a variant in Shopify removes only that variant from MCSL — parent and remaining variants untouched	hamburger menu > Products > expand parent: deleted variant absent, others unchanged
products/update webhook returns 2xx on first delivery and on retry (idempotent)	Request log: 2xx response for each products/update event; Products list still shows exactly one row per product ID
products/create webhook still creates exactly one new row (regression check)	hamburger menu > Products: new product appears once, keyed by new Shopify product ID
Bulk variant edits (rename multiple variants in one save) all apply without duplication	hamburger menu > Products > expand parent: all renamed variants reflect new titles; total variant count unchanged
Request log node: webhook topic	products/update or products/create as appropriate
Request log node: product identifier	Shopify product ID (not product title) used as the upsert key
Request log node: variant identifier	Shopify variant ID used as the upsert key for variant-level changes
HTTP response in log	2xx for all product sync events

Merchant-Safe Explanation

When you rename a product or variant in Shopify, the PluginHive app now recognises that it is the same product — just with a new name — and updates your existing entry instead of creating a second one. This keeps your product list in the app clean and accurate, so your shipping rates, packaging rules, and labels always reference the right product without any manual cleanup needed on your end.

Common Questions & Troubleshooting

› "I still see the old product name as a separate row."

Check first: confirm the store is on MCSL SL 382 or later. If not, the fix is not deployed — escalate for a release update. If the build is correct, check the request log for a products/update event around the time of the rename and confirm it returned 2xx.

› "The duplicate disappeared but now my shipping rate rule references a missing product."

The stale duplicate row has been removed. Ask the merchant to review **hamburger menu > Settings > Shipping Rates / Automation** and **hamburger menu > Settings > Packaging** and update any rules that previously pointed to the old duplicate row to reference the current product.

› "A variant I deleted in Shopify is still showing in MCSL."

Check the request log for a products/update event at the time of deletion. If no webhook was received, the webhook may have failed delivery — check Shopify Admin > Settings > Notifications > Webhooks. If the webhook was received and returned 2xx but the variant persists, escalate with log evidence and a screenshot of the Products list.

› "I renamed multiple variants at once and some still show the old names."

Check the request log for the products/update event covering the bulk edit. Confirm all variant IDs were included in the payload and the response was 2xx. If some variants did not update, escalate with the full request/response log for that event.

› "A brand-new product I created is not appearing in MCSL."

This is a separate products/create flow. Check the request log for a `

ZI-566 - From SL: ZI-566 — Add TForce Freight as supported carrier [#389467]

Support Guide: ZI-566 — Add TForce Freight as Supported Carrier

From SL: ZI-566 — Add TForce Freight as supported carrier [#389467]

Feature Summary

This card adds **TForce Freight** as a new supported LTL (less-than-truckload) carrier in the PluginHive Multi-Carrier Shipping Label (MCSL) app on **Shopify**. Prior to this change, TForce Freight was not available in the carrier selection list, meaning merchants shipping palletized or freight-weight orders had no TForce option within MCSL. With this release, US-based merchants can configure TForce Freight credentials, fetch LTL rates, generate freight labels (with PRO/tracking number), and track shipments — all from within the MCSL app alongside existing parcel carriers such as FedEx and UPS. This is a net-new carrier integration; no existing carrier behaviour is modified.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Feature should be available to all merchants on the qualifying MCSL release (SL MCSL 382 iteration backlog); confirm the store is on the correct app version
TForce Freight credentials required	Merchant must have a valid TForce account number, API user, and API password — sandbox or live; confirm credentials are available before troubleshooting rate/label failures
US-origin stores only	Store origin address must be a US address; non-US origins are expected to return an unsupported-origin error — confirm origin address in Shopify Admin > Settings > Locations
LTL/freight-eligible order required	Orders must meet freight criteria (palletized packaging, appropriate weight/dimension thresholds, or merchant override); confirm order packaging is configured as freight/pallet type in MCSL
FedEx / UPS regression scope	FedEx and UPS are the existing carriers on the test/reference store; confirm these continue to function normally after TForce is added
Shopify platform	This integration is validated on Shopify; confirm the merchant's store platform before applying this guide

Step-by-Step Support Walkthrough

Part A — Carrier Configuration

1. Open the MCSL app from Shopify Admin.

In Shopify Admin, go to **Apps > PH Multi Carrier Shipping Label App** to open the MCSL dashboard.

2. Navigate to the Carriers settings.

Click the **hamburger menu > Carriers**. Confirm the Carriers list loads and existing carriers (e.g. FedEx, UPS) are shown as Active.

3. Verify TForce Freight appears in the Add Carrier list.

Click **Add Carrier**. In the carrier selection dialog, confirm **TForce Freight** is present with its logo. If TForce Freight is not listed, the store may not be on the correct app version — confirm the release and escalate if needed.

4. Configure TForce Freight credentials.

Select **TForce Freight** from the list. The configuration form should prompt for:

- › **Account Number**
- › **API User**
- › **API Password**

Enter valid credentials (sandbox or live as applicable). Click **Test Connection**.

- › **Request/log fields to verify:** Confirm the test-connection request is sent to the TForce API endpoint and that a success response is returned. Check **hamburger menu > Settings > Request**

Log (or equivalent log path) if the UI result is ambiguous.

5. Save the carrier and confirm persistence.

After a successful Test Connection, click **Save**. Reload the Carriers page and confirm TForce Freight appears as **Active** in the carrier list with the account number masked/partially displayed.

6. Verify invalid credentials are rejected.

If testing the negative path: enter known-bad credentials and click **Test Connection**. Confirm a clear validation error is shown (inline or toast) and that the merchant cannot Save until credentials are corrected.

- › **Request/log fields to verify:** Check the request log to confirm the failed test-connection request and the error response from TForce are captured.

Part B — Rate Fetch

7. Open a freight-eligible order in MCSL ORDERS.

Go to the **ORDERS** tab in MCSL. Open an order that has palletized/freight packaging configured (e.g. 1 pallet, ~200 lb, appropriate dimensions). If no such order exists, confirm packaging is set up under **hamburger menu > Products** or the order's packaging settings before proceeding.

8. Fetch rates for the order.

Inside the order, click **Prepare Shipment** (or **Get Rates** depending on the MCSL UI version). Confirm TForce Freight rate options appear in the **Rate Summary** section alongside any other active carriers.

Each TForce rate entry should display:

- › Service name
- › Total rate amount
- › Transit days (if returned by TForce API)
- › **Request/response nodes to verify:** In **Order Summary > View Log**, confirm the outbound rate request contains the correct origin (US address), destination, weight, dimensions, and packaging type. Confirm the TForce rate response contains at least one valid service/rate entry.

9. Verify non-US origin handling (if applicable).

If the store origin is non-US, confirm that attempting to fetch TForce rates returns a clear **"US-origin only"** or equivalent unsupported-origin error message — not a silent failure or generic API error.

- › **Request/log fields to verify:** Check the request log to confirm the error is surfaced from the TForce API or caught at the MCSL validation layer.

Part C — Label Generation

10. Select a TForce rate and generate a label.

From the Rate Summary, select a TForce Freight rate. Click **Generate Label**.

Confirm the following in **Order Summary > Label Summary**:

- › A TForce Freight label PDF is generated and downloadable
- › The **PRO number** (TForce tracking number) is displayed on the Order Summary
- › **Request/response nodes to verify:** In **Order Summary > View Log**, confirm the label request payload includes the selected service, shipment details, and credentials. Confirm the response contains the PRO number and label data (PDF or URL).

11. Verify the tracking link.

In **MCSL ORDERS > Order Summary**, click the TForce PRO tracking number. Confirm:

- › A new browser tab opens
- › The URL contains the PRO number as a path or query segment
- › The destination is the TForce tracking portal

Part D — Regression Check

12. Confirm FedEx and UPS are unaffected.

With TForce Freight now active, open a standard domestic parcel order in the **ORDERS** tab. Fetch rates and confirm FedEx and UPS rates are returned as expected. Generate a label for each and confirm successful label generation. No regression should be introduced by adding TForce Freight.

Expected Behaviour

What support should observe	How to confirm
TForce Freight appears in the Add Carrier dialog with its logo	hamburger menu > Carriers > Add Carrier — TForce Freight is listed
Test Connection succeeds with valid TForce credentials	Success message shown in UI; success response visible in request log
Invalid credentials produce a clear, actionable error and block Save	Error toast or inline error shown; Save button inactive until corrected; failed request visible in log
TForce Freight saved and shown as Active in Carriers list	Carriers list after Save and page reload shows TForce Freight as Active with masked account
TForce Freight rate options appear in Rate Summary for a freight-eligible US order	Rate Summary shows TForce service name, amount, and transit days (if returned)
Rate request payload contains correct US origin, destination, weight, dims, and packaging	Order Summary > View Log — outbound rate request node
Rate response contains at least one valid TForce service/rate entry	Order Summary > View Log — inbound rate response node
TForce Freight label PDF is generated and downloadable	Order Summary > Label Summary — label PDF link present
PRO/tracking number is displayed on Order Summary	PRO number visible in Order Summary after label generation

What support should observe	How to confirm
Label request/response captured in View Log	Order Summary > View Log — label request and response nodes present
TForce tracking link opens in a new tab with PRO number in URL	Clicking PRO number in Order Summary opens TForce tracking portal in new tab
Non-US origin returns a clear unsupported-origin error, not a silent failure	UI error message shown; error captured in request log
FedEx and UPS rates and labels continue to work without regression	Rate Summary and Label Summary for parcel orders show FedEx/UPS results unchanged

Merchant-Safe Explanation

TForce Freight is now available as a supported carrier in your PluginHive Multi-Carrier Shipping Label app. If you ship LTL freight from a US location, you can add TForce Freight under **Settings > Carriers**, enter your TForce account credentials, and start fetching freight rates and generating labels directly from your MCSL app — right alongside your existing carriers like FedEx and UPS. Once a label is generated, your TForce PRO tracking number will appear in the order summary and link directly to the TForce tracking portal. Please note that TForce Freight is supported for US-origin shipments only.

Common Questions & Troubleshooting

- › **TForce Freight not visible in Add Carrier list:** Confirm the store is running the MCSL release that includes SL MCSL 382 / ZI-566. If the carrier is still missing after confirming the version, escalate with the store URL and app version.
- › **Test Connection fails with valid credentials:** Check the request log (hamburger menu > Settings > Request Log or Order Summary > View Log) for the raw error response from TForce. Confirm the merchant is using the correct credential type (account number vs. API user vs. API password) — these are distinct fields.

ZI-567 - From SL: ZI-567 — Add configurable buffer days to DHL Express delivery estimates [#392107]

Support Guide: ZI-567 — DHL Express Configurable Buffer Days

From SL: ZI-567 — Add configurable buffer days to DHL Express delivery estimates [#392107]

Feature Summary

This change adds a configurable **buffer days** field to the DHL Express carrier settings in MCSL on Shopify. When set, the buffer value is added to the transit days returned by DHL Express before the estimate is displayed at checkout. The feature allows merchants to pad delivery estimates to account for pick/pack lead time, reducing customer complaints about late deliveries. The buffer is scoped to **DHL Express** (REST and legacy); FedEx and Australia Post are not affected. The allowed input range is **0–10 whole days**, with 0 acting as a no-op.

Toggles & Prerequisites

Toggle / config note	What support should confirm
Buffer Days field (Carrier settings > Other Details, DHL Express)	Confirm the field is visible in the DHL Express carrier config. Valid range: integers 0–10 only. Negative values, decimals, and values > 10 should be blocked at save with a validation error.
DHL Express (REST and legacy) carrier account	Feature applies to DHL Express REST and legacy. Confirm the store is using the DHL Express integration.
FedEx / Australia Post	These carriers must not be affected by the DHL buffer setting. Confirm their transit values are unchanged when DHL buffer > 0.
Release prerequisite	Card is in SL MCSL 382 iteration backlog. Confirm the store is on a build that includes this release before troubleshooting.
Test/reference store	QA verified on <code>dhl-mcsl-automation.myshopify.com</code> and <code>dhl-express-381.myshopify.com</code> (DHL Express REST). Use these as reference if needed.

Step-by-Step Support Walkthrough

1. Confirm the app build includes this feature.

Open the MCSL app on the merchant's Shopify store. Confirm the installed version is on or after the SL MCSL 382 release. If the buffer field is not visible in step 2, the store may be on an older build — escalate for deployment confirmation.

2. Locate the Buffer Days field in DHL Express carrier settings.

Navigate to **Hamburger menu > Carriers**, then open the **DHL Express** carrier configuration.

Go to the **Other Details** section within the DHL Express carrier settings.

Confirm the **Buffer Days** field is present and accepts a numeric input.

3. Set a test buffer value and save.

Enter a buffer value between **1 and 9** (e.g., 2) to produce a clearly observable change.

Click **Save** and confirm no validation error appears.

- › To verify boundary enforcement: enter 11 and confirm a validation error is shown and the previous valid value is retained on page reload.
- › To verify negative/decimal blocking: enter -1 or 1.5 and confirm a validation error is shown.

4. Fetch a DHL Express rate for an international order and check the displayed transit.

Go to the **ORDERS** tab, open an order eligible for DHL Express international shipping, and open **Prepare Shipment / Generate Label**.

Alternatively, trigger a checkout rate fetch from the storefront using a DHL-eligible international destination.

Observe the transit days displayed for the DHL Express rate.

› **Expected:** If the DHL API returns `TotalTransitDays = 5` and `buffer = 2`, the displayed transit should be **7 days**.

› **Expected (buffer = 0):** Displayed transit matches the raw DHL API value — no change.

5. Verify the buffer in the request log.

Navigate to **Hamburger menu > Settings > Request Log** (or the equivalent log viewer in the app).

Locate the most recent DHL Express rate request for the order tested in step 4.

› - Request/log fields to verify: `TotalTransitDays` (raw value from DHL response) and the final composed transit value sent to the storefront. Confirm: `final transit = TotalTransitDays + buffer days`.

6. Verify the edge case: missing or zero `TotalTransitDays`.

If the DHL response returns `TotalTransitDays = 0` or the field is absent, and `buffer = 3`, the displayed transit should be **3 days** (buffer alone) with no NaN, undefined, or blank value shown.

› - Request/log fields to verify: Confirm the log records the fallback behaviour and does not show an error or null transit value.

7. Verify the buffer does not affect FedEx or Australia Post (regression check).

With DHL buffer set to a non-zero value (e.g., 3), fetch rates for the same or a comparable order that also returns FedEx and Australia Post rates.

Confirm FedEx and Australia Post transit values match the raw carrier API responses — no buffer applied.

› - Request/log fields to verify: FedEx and Australia Post transit day values in the log should be unchanged from their respective carrier responses.

8. Verify immediate effect at checkout without restart.

With a DHL buffer already saved (e.g., 1), note the current checkout estimate for a DHL rate.

Change the buffer to 3 and save.

Have the customer (or test user) refresh the checkout page.

Confirm the updated buffer is reflected in the new DHL delivery estimate immediately — no app or server restart should be required.

Expected Behaviour

What support should observe	How to confirm
Buffer Days field is visible in DHL Express carrier settings under Other Details	Hamburger menu > Carriers > DHL Express > Other Details
Buffer = 0 leaves DHL transit unchanged from the raw API value	Rate displayed at checkout or in Prepare Shipment matches <code>TotalTransitDays</code> from DHL response; request log confirms no addition
Buffer = N (1–10) adds exactly N days to the displayed DHL transit	Displayed transit = <code>TotalTransitDays + buffer</code> ; confirmed in request log

What support should observe	How to confirm
Buffer = 0 fallback when TotalTransitDays is missing/zero	Displayed transit = buffer value only; no NaN, undefined, or blank; log records fallback
Values > 10, negative values, and decimals are blocked	Validation error shown on save; page reload retains previous valid value
FedEx and Australia Post transits are unchanged	Side-by-side rate comparison shows only DHL transit is buffered; FedEx and Australia Post match raw carrier responses

ZI-568 - From SL: ZI-568 — FedEx Regional Economy address line duplication on labels [#389508]

Support Guide: ZI-568 — FedEx Regional Economy Address Line Duplication on Labels

From SL: ZI-568 — FedEx Regional Economy address line duplication on labels [#389508]

Feature Summary

This fix resolves a bug where FedEx Regional Economy labels generated through PH Multi Carrier Shipping Label (MCSL) on Shopify displayed duplicated street address lines — both on the printed label PDF and in the outbound FedEx API request payload. The root cause was MCSL's internal address model storing address data under two naming conventions (`address_1/address_2` and `addressLine1/addressLine2`); when both fields were populated — particularly after address validation rewrote address fields — the `streetLines` array builder emitted the same value more than once. The fix introduces deduplication logic in the address build path before the FedEx `streetLines` array is emitted. DHL shares this address build path and must be regression-verified. Two related open issues remain in scope as separate cards: shipper address line 1 still repeating three times in the label request, and commercial invoice duplication for both From and To addresses.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Fix is code-level; no merchant-facing toggle to enable or disable
FedEx Regional Economy service must be configured	Confirm FedEx carrier is connected under hamburger menu > Carriers and Regional Economy is an active service
DHL carrier configured (regression check)	Confirm DHL is also connected if the merchant reports DHL label issues post-fix
Address validation feature (if active)	Confirm whether the store has address validation/correction enabled — this is the highest-risk path for triggering the original bug (TC-3)
Release: SL MCSL 382 iteration backlog	Confirm the store is running an app version that includes this fix; check app version in hamburger menu > Settings or confirm from release deployment records
Platform: Shopify	Fix was developed and QA-verified on Shopify; validate on Shopify Admin + MCSL ORDERS tab

Toggle / config note	What support should confirm
Two related open issues exist	Shipper address line 1 still repeating thrice (separate card: trello.com/c/TV8fKLPz) and commercial invoice From/To duplication (separate card: trello.com/c/zxpoHZjw) — do not conflate with this fix

Step-by-Step Support Walkthrough

1. Confirm the carrier and service are configured correctly on the store.

- › In MCSL, go to **hamburger menu > Carriers**.
- › Confirm FedEx is connected and **FedEx Regional Economy** is listed as an active service.
- › If the merchant also uses DHL, confirm DHL is connected here as well.

2. Identify a test or affected order with a known address structure.

- › In **Shopify Admin > Orders**, locate the order the merchant reported, or use a test order.
- › Note the exact values in the shipping address: `address_1`, `address_2` (if any), and whether the address appears to have been modified by address validation.
- › The highest-risk scenario is an order where address validation was triggered — both `address_1` and `addressLine1` may have been populated with the same value.

3. Open the order in MCSL and navigate to label generation.

- › In MCSL, go to the **ORDERS** tab.
- › Locate the order and open it.
- › Click **Prepare Shipment** and select **FedEx Regional Economy** as the service.
- › Proceed to **Generate Label**.

4. Inspect the outbound FedEx request payload for `streetLines`.

- › After label generation (or on failure), go to **hamburger menu > Settings** (or the **Request Log** path available in the app) and open the request log for this label attempt.
- › Alternatively, from the order view, navigate to **Order Summary > Label Summary > View Log**.
- › - Request nodes to verify: `streetLines` array — confirm the array contains only unique, non-empty entries; a single-line address should have exactly 1 entry; a two-line address should have exactly 2 entries with no repeated values.

5. Verify the generated label PDF.

- › Download the label PDF from the MCSL ORDERS tab (open the order > Label Summary > Download Label).
- › Visually inspect the **To address block** on the label.
- › Confirm the street address line(s) appear **exactly once** — no repeated lines stacked on top of each other.
- › For a single-line address (e.g., "123 Main St"), only one street line should be visible.
- › For a two-line address (e.g., "123 Main St" + "Apt 4B"), both lines should appear once each.

6. Test the address validation edge case (TC-3 — the primary bug reproduction path).

- › Use a Shopify order where address validation is active and has rewritten the address fields.

- › Generate a FedEx Regional Economy label for this order.
- › Open the request log and confirm `streetLines` contains the address value **only once**, even if both `address_1` and `addressLine1` were populated internally.
- › - Request nodes to verify: `streetLines` – confirm deduplication is applied; value should not appear more than once regardless of which internal field convention populated it.

7. Test with an empty or whitespace-only `address_2`.

- › Use or create a Shopify test order where `address_2` is blank or contains only spaces.
- › Generate a FedEx Regional Economy label.
- › Open the request log and confirm `streetLines` has **exactly 1 entry** — no empty or whitespace-only second entry is present.
- › - Request nodes to verify: `streetLines` array length – should be 1 when `address_2` is empty or whitespace.

8. Regression check: DHL label address rendering.

- › Using the same test order (single-line or two-line address), generate a **DHL** label instead of FedEx.
- › Open the DHL request log and inspect the address payload (equivalent `streetLines` or address line fields for DHL).
- › Confirm DHL address lines are also rendered exactly once — the shared address build path must not have been broken by the FedEx fix.
- › - Request/log fields to verify: DHL address line fields in the outbound request – confirm no duplication introduced by the fix.

9. Void and regenerate to confirm stability.

- › Void the FedEx Regional Economy label from the MCSL ORDERS tab.
- › Regenerate the label for the same order.
- › Compare both label PDFs and both request logs side by side.
- › - Request nodes to verify: `streetLines` in both label generation attempts – values must be identical and non-duplicated across both runs.

10. Cross-check known open issues — do not conflate.

- › If the merchant reports the **shipper/From address** line still repeating, this is a **separate open issue** tracked at trello.com/c/TV8fKLPz and is not resolved by this card.
- › If the merchant reports **commercial invoice** address duplication (both From and To), this is tracked at trello.com/c/zxpoHZjw and is also a separate open item.
- › Document clearly which symptom the merchant is experiencing before escalating.

Expected Behaviour

What support should observe	How to confirm
FedEx Regional Economy label PDF shows the To street address line(s) exactly once	Download label PDF from MCSL ORDERS tab > Label Summary; visually inspect address block

What support should observe	How to confirm
streetLines array in FedEx request contains only unique, non-empty entries	Open request log via Order Summary > Label Summary > View Log; inspect streetLines array
Single-line address produces streetLines with exactly 1 entry	Request log: streetLines array length = 1
Two-line address produces streetLines with exactly 2 entries, no repeats	Request log: streetLines = ["address_1 value", "address_2 value"] — no value appears twice
Address validation path (both address_1 and addressLine1 populated) produces deduplicated streetLines	Request log: streetLines contains the address value once only, even when both internal fields were set
Empty or whitespace-only address_2 is omitted from streetLines	Request log: streetLines array length = 1; no empty string or whitespace entry present
DHL label address rendering is unaffected by this fix	DHL request log and DHL label PDF show address lines exactly once — no regression
Regenerating the label produces identical streetLines output	Two request logs for the same order show matching streetLines arrays
Shipper/From address line repetition is not resolved by this card	If From address still repeats, confirm this is the separate open card (TV8fKLPz) and escalate accordingly
Commercial invoice address duplication is not resolved by this card	If commercial invoice shows duplicated From/To, confirm this is the separate open card (zxpoHZjw)

Merchant-Safe Explanation

We identified and fixed an issue where FedEx Regional Economy shipping labels were showing the delivery address street line more than once — both on the printed label and in the data sent to FedEx. This happened because our app was reading address information from two internal sources and combining them without checking for duplicates first. The fix ensures that each address line appears exactly once on your FedEx Regional Economy labels. If you are still seeing address repetition on the **sender address** or on

ZI-570 - From SL: ZI-570 — Shipping rate automation fails for Guam and Marshall Islands zones [#391779]

Support Guide: ZI-570 — Shipping Rate Automation for Guam and Marshall Islands

From SL: ZI-570 — Shipping rate automation fails for Guam and Marshall Islands zones [#391779]

Feature Summary

Customer-reported issue: On Shopify stores, Shipping Rate Automation zone matching was not triggering for **Guam (GU)** and **Marshall Islands (MH)**. As reported by the customer:

- › The automation rule did not trigger for Guam and Marshall Islands.
- › The automation summary was blank.
- › No flat rate was returned at checkout, and customers saw **"Shipping not available."**

Technical / root-cause impact: The underlying cause was address-normalization handling for these territories — covering both the `country=US, state=GU` and `country=GU` address forms. The same normalization gap also produced downstream carrier errors that affected carrier rate retrieval and label generation: UPS 111286 ("GU is not a valid state for the specified shipment") for Guam and UPS 111100 ("The requested service is invalid from the selected origin") for Marshall Islands.

Fix scope: This change corrects territory address normalization so automation zone matching, rate selection, and label generation work correctly. The fix **primarily addresses Guam (GU) and Marshall Islands (MH)**. Per the ticket notes, **Puerto Rico (US-PR) and AA military addresses were already working** — the fix **validates continued compatibility** with PR and AA rather than introducing new behaviour for them. The change is part of MCSL 382 and affects the shipping rate automation engine on Shopify.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Fix is applied at the code/config level; no merchant-facing toggle to enable
Shopify store on MCSL 382 or later	Confirm the store is on the correct app release before troubleshooting
FedEx and/or UPS carrier connected	Confirm at least one of these carriers is active under hamburger menu > Carriers
Shipping zone configured with Guam and/or Marshall Islands	Confirm the zone exists and contains the correct territory under hamburger menu > Settings > Shipping Rates / Automation
Automation rule configured for the territory zone	Confirm a rate rule is attached to the Guam/MH zone (e.g., \$20 territory rate)
Address form variation (US-wrapped vs standalone)	Confirm whether the store's checkout returns <code>country=US, state=GU</code> or <code>country=GU</code> — both should now be handled
Customer/test platform	Shopify

Step-by-Step Support Walkthrough

1. Confirm the app release on the merchant's Shopify store.

- › In Shopify Admin > Apps > PH Multi Carrier Shipping Label App, check the app version or release identifier.
- › Confirm the store is running MCSL 382 or later. If not, this fix is not yet deployed and escalation is required before further troubleshooting.

2. Confirm carrier connections for FedEx and UPS.

- › Navigate to hamburger menu > Carriers in the MCSL app.
- › Confirm FedEx and/or UPS accounts are connected and active.
- › Note which carrier the merchant is using for the affected territory shipments.

3. Confirm the shipping zone configuration for Guam and/or Marshall Islands.

- › Navigate to hamburger menu > Settings > Shipping Rates / Automation (or the equivalent Automation/Rate Rules section).
- › Confirm a shipping zone exists that includes Guam (GU) and/or Marshall Islands (MH) as destinations.
- › Confirm the zone is not empty and that the correct territory is listed — not just a mainland US zone.
- › If a combined zone is configured (e.g., PR + GU + MH + AA), confirm all four territories are present.

4. Confirm the automation rate rule is attached to the territory zone.

- › Within the same Automation/Rate Rules section, confirm a rate rule (e.g., a \$20 territory rate) is linked to the Guam/MH zone.
- › Confirm the rule's match condition (Match All or Match Any) is set as intended.
- › Confirm no duplicate rules exist that could cause cross-firing between mainland and territory zones.

5. Reproduce or verify the rate return for a Guam order.

- › In Shopify Admin > Orders, locate a test or live order with destination ZIP 96910 (Guam).
- › Open the order in the MCSL app via the ORDERS tab, then open the order and navigate to Prepare Shipment / Generate Label.
- › Confirm rates are returned for the configured carrier (FedEx and/or UPS).
- › Confirm the automation summary shows Guam as the matched destination and the correct rate is selected.
- › - **Request/log fields to verify:** UPS error code 111286 should no longer appear; confirm absence of "GU is not a valid state" in the request log. Check hamburger menu > Settings > Request Log for the relevant order.

6. Reproduce or verify the rate return for a Marshall Islands order.

- › Locate a test or live order with destination ZIP 96960 (Marshall Islands).
- › Open the order in the ORDERS tab > Prepare Shipment / Generate Label.
- › Confirm rates are returned. Prior to the fix, UPS returned no rates for MH; this should now be resolved.
- › - **Request/log fields to verify:** UPS error code 111100 ("The requested service is invalid from the selected origin") should no longer appear for valid MH destinations. Verify in hamburger menu > Settings > Request Log.

7. Verify address normalisation for both Guam address forms.

- › Confirm the fix handles both address variants:
 - › country=US, state=GU, ZIP 96910 (Shopify checkout wraps Guam as a US state)
 - › country=GU, ZIP 96910 (Guam as a standalone country)
- › Both forms should match the Guam automation rule and return the same rate.
- › If the merchant reports rates working in one form but not the other, capture the exact address payload from the request log.
- › - **Request nodes to verify:** country, state, and postalCode fields in the outbound carrier request log to confirm normalisation is applied before the request is sent to UPS/FedEx.

8. Verify mutual exclusivity of mainland and territory rules.

- › If the merchant has both a mainland US rate rule and a territory rate rule configured, confirm only the correct rule fires for each destination.
- › Test a mainland ZIP (e.g., 90210) and confirm no territory rule appears in the automation log.
- › Test a Guam ZIP (96910) and confirm no mainland rule appears.
- › - **Request/log fields to verify:** Automation summary log — confirm only one rule is listed per order, with no duplicate rates at checkout.

9. Verify label generation for Guam after the fix.

- › In the ORDERS tab, open a Guam-destined order and attempt to Generate Label.
- › Confirm the label generates successfully without the 111286 UPS error.
- › Confirm the generated label/tracking number is returned and visible in the order.

10. Check for invalid postal code handling (negative case).

- › If a merchant reports no rates for a Guam or MH order, confirm the ZIP code is valid:
- › Guam: 96910–96932 range
- › Marshall Islands: 96960, 96970
- › An invalid ZIP should return a clear no-match or address validation message — not a 500 error or silent failure.
- › - **Request/log fields to verify:** Rate log entry for the order — confirm a clean error message is returned, not an unhandled exception.

Expected Behaviour

What support should observe	How to confirm
Rates returned for Guam (ZIP 96910) via UPS and/or FedEx	ORDERS tab > open order > Prepare Shipment — rates list is populated
Rates returned for Marshall Islands (ZIP 96960) via UPS and/or FedEx	ORDERS tab > open order > Prepare Shipment — rates list is populated
UPS error 111286 ("GU is not a valid state") no longer appears	hamburger menu > Settings > Request Log — error absent for Guam orders
UPS error 111100 ("The requested service is invalid from the selected origin") no longer appears for valid MH destinations	hamburger menu > Settings > Request Log — error absent for MH orders
Both country=US, state=GU and country=GU address forms match the Guam automation rule	Automation summary in ORDERS tab shows Guam as matched destination for both address forms
Automation summary correctly identifies Guam or MH as the matched destination	Order Summary / automation log in ORDERS tab lists the correct territory and rate
Mainland US orders (e.g., ZIP 90210) are unaffected — only mainland rule fires	Automation log shows no territory rule triggered for mainland orders

What support should observe	How to confirm
Combined territory zone (PR + GU + MH + AA) matches all four destinations	Each territory test order shows the correct rate in the automation summary
No duplicate rates returned at checkout for territory destinations	Storefront checkout / Shopify Admin > Orders — single rate displayed per zone match
Label generation succeeds for Guam orders	Generated label and tracking number visible in ORDERS tab after Generate Label action
Invalid Guam or MH postal codes return a clean no-match message, not a 500 error	Request Log — clean error entry, no unhandled exception

Merchant-Safe Explanation

Previously, for shipments to **Guam** and the **Marshall Islands**, the shipping rate automation was not matching the destination — so no flat rate was returned and customers saw "Shipping not available" at checkout. Behind the scenes, the address was being sent in a format the carrier did not accept, which also caused rate and label errors. This has been corrected: the app now properly formats these territory addresses, so the automation rule matches and rates and labels work as expected for Guam and the Marshall Islands. Puerto Rico (PR) and military (AA) addresses were already working and continue to work as before. No changes are needed on your end — the fix is applied automatically.

Common Questions & Troubleshooting

- › **Check first:** Confirm the store is on MCSL 382 or later. If the app has not been updated, the fix is not active and no amount of configuration changes will resolve the issue.
- › **Check first:** Confirm the shipping zone in hamburger menu > Settings > Shipping Rates / Automation actually

ZI-571 - From SL: ZI-571 — WooCommerce orders not auto-importing due to webhook user permissions [#389228]

Support Guide: ZI-571 — WooCommerce Webhook User Permissions Fix

From SL: ZI-571 — WooCommerce orders not auto-importing due to webhook user permissions [#389228]

Feature Summary

Prior to plugin v2.1.9, WooCommerce webhooks created by the PluginHive MCSL plugin were tied to the specific WordPress admin user who originally registered the plugin. If that user was later deleted — a routine admin change — the webhooks became orphaned, stopped firing, and order/product changes in WooCommerce silently stopped auto-importing into the MCSL app. This fix, delivered in plugin v2.1.9, decouples webhook ownership from any individual admin user so that the integration remains stable regardless of admin account changes. Affected entities are WooCommerce orders and products on the

auto-import path. This fix is **not carrier-specific** — it applies to the WooCommerce auto-import integration regardless of carrier. A label-generation regression check with any configured carrier is included.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	No gating required — behaviour is version-driven
Plugin version prerequisite	Merchant must be on PluginHive WooCommerce plugin v2.1.9 or later for the fix to apply
Recovery step required for upgraded sites	Merchants upgrading from v2.1.8 (or earlier) with already-broken webhooks must delete all existing WooCommerce webhooks and trigger a Re-sync from the plugin Settings page — upgrade alone is not sufficient
WooCommerce Webhooks page access	Confirm merchant or support has access to WooCommerce Admin > Settings > Advanced > Webhooks to inspect webhook status
Carrier account prerequisite (regression check)	Any carrier configured in MCSL can be used for the TC-6 label-generation regression check — this fix is not carrier-specific
Customer/test platform	WooCommerce
Release	SL MCSL 382 — Iteration backlog

Step-by-Step Support Walkthrough

Confirm plugin version first — all steps depend on this.

1. Verify the installed plugin version on the WooCommerce site.

- › In WordPress Admin, go to **Plugins > Installed Plugins** and locate the PluginHive shipping plugin.
- › Confirm the version shown is **v2.1.9 or later**.
- › If the merchant is still on v2.1.8 or earlier, the fix is not present. Direct them to upgrade before proceeding.

2. Check the current state of WooCommerce webhooks.

- › In WordPress Admin, navigate to **WooCommerce > Settings > Advanced > Webhooks**.
- › Review the webhook list for any entries with status **Inactive**, **Disabled**, or a missing/deleted owner.
- › On a broken v2.1.8 site, orphaned webhooks will appear here after the registering admin is deleted.
- › On a healthy v2.1.9 site (fresh install or post-resync), all PluginHive webhooks should show **Active** status with a non-user-tied owner.
- › - Request/log fields to verify: Webhook Status column (Active vs Inactive/Disabled), Delivery URL (should point to the PluginHive/MCSL endpoint), Owner/Created By field

3. For merchants upgrading from v2.1.8 with broken webhooks — execute the recovery flow.

- › Confirm the merchant has already upgraded to v2.1.9 (Step 1).

- › In **WooCommerce > Settings > Advanced > Webhooks**, delete **all** existing PluginHive webhooks.
- › In the PluginHive plugin, go to **WordPress Admin > PluginHive MCSL plugin Settings page** and trigger **Re-sync** (confirm exact button label from live store/card context).
- › After re-sync completes, return to **WooCommerce > Settings > Advanced > Webhooks** and confirm new webhooks have been created with **Active** status.
- › This recovery step is mandatory for upgraded sites — upgrade alone does not repair orphaned webhooks.

4. Verify auto-import is working after the fix or recovery.

- › In WooCommerce Admin, create or edit a test order (or product).
- › Wait within the webhook propagation window (confirm expected window from live store/card context).
- › In the MCSL app, open the **ORDERS** tab and confirm the new or updated order appears in the order grid without any manual import action.
- › If the order does not appear, check Step 2 again — webhooks may still be inactive.

5. Verify the Admin-A deletion regression scenario (if applicable to the merchant's report).

- › On a v2.1.9 site, confirm that deleting the admin user who originally registered the plugin does **not** cause webhooks to go inactive.
- › After the admin deletion, return to **WooCommerce > Settings > Advanced > Webhooks** and confirm all PluginHive webhooks remain **Active**.
- › Edit a WooCommerce order and confirm it auto-imports into the MCSL **ORDERS** tab as expected.

6. Verify manual import and label generation are unaffected (regression check).

- › In the MCSL app, open the **ORDERS** tab and manually import a WooCommerce order using the manual import option.
- › Open the imported order and navigate to **Prepare Shipment / Generate Label**.
- › Confirm rates are returned and a label can be generated without errors using any configured carrier (this fix is not carrier-specific).
- › Check **Hamburger menu > Settings > Request Log** for any errors introduced by the webhook fix.
- › - Request/log fields to verify: No error entries tied to the webhook ownership change; the rate request and label response should be clean

Expected Behaviour

What support should observe	How to confirm
WooCommerce webhooks show Active status on v2.1.9	WooCommerce Admin > Settings > Advanced > Webhooks — Status column reads Active for all PluginHive webhooks
Webhook owner is not tied to a specific deleted admin user	Webhooks page shows no orphaned/missing owner after Admin-A deletion on v2.1.9
Order edits in WooCommerce auto-import into MCSL without manual action	Edit a WooCommerce order; confirm it appears in MCSL ORDERS tab within the propagation window

What support should observe	How to confirm
Product updates in WooCommerce auto-import into MCSL	Edit a WooCommerce product; confirm update is reflected in MCSL Hamburger menu > Products
Recovery flow restores auto-import on upgraded sites	After delete-and-resync, new Active webhooks appear and subsequent order edits auto-import correctly
Upgrade alone (v2.1.8 → v2.1.9) without resync does not automatically repair broken webhooks	Webhooks remain inactive until the merchant manually deletes and re-syncs
Manual order import continues to work on v2.1.9	Order imported via MCSL ORDERS tab manual import flow appears correctly
Label generation is unaffected (any configured carrier)	Label generated successfully from Prepare Shipment / Generate Label ; no errors in Request Log
No errors in Request Log related to webhook ownership	Hamburger menu > Settings > Request Log — no new error entries after fix deployment

Merchant-Safe Explanation

WooCommerce uses webhooks to automatically send order and product updates to the PluginHive shipping app. In older versions of the plugin, these webhooks were linked to the specific admin account that set up the plugin. If that admin account was ever removed, the webhooks would stop working and new orders or changes would no longer appear in the app automatically. Version 2.1.9 of the plugin fixes this by removing that dependency on a specific admin account. If you have already upgraded and are still seeing the issue, you will need to delete your existing webhooks from **WooCommerce > Settings > Advanced > Webhooks** and then use the Re-sync option in the plugin settings — this recreates the webhooks correctly and restores automatic order importing.

Common Questions & Troubleshooting

- › **Check first:** Confirm the merchant is on plugin v2.1.9 or later. If not, the fix is not present and the upgrade is the first action.
- › **Check first:** Go to **WooCommerce > Settings > Advanced > Webhooks** and confirm all PluginHive webhooks show **Active** status. Inactive or orphaned webhooks are the direct cause of failed auto-import.
- › **If the merchant upgraded but auto-import is still broken:** The recovery flow (delete all webhooks + Re-sync from plugin Settings) is mandatory for sites that were already in a broken state before the upgrade. Confirm the merchant has completed both steps, not just the upgrade.
- › **If webhooks are Active but orders still do not auto-import:** Confirm the webhook Delivery URL points to the correct PluginHive/MCSL endpoint. A URL mismatch after a site migration or domain change can cause silent failures unrelated to this fix.
- › **If the merchant reports the issue started after removing an admin user:** This is the exact scenario this fix addresses. Confirm plugin version and walk through the recovery flow.
- › **Inspect logs when:** Auto-import is not working despite Active webhooks — check **Hamburger menu > Settings > Request Log** for delivery failures or authentication errors on the webhook

endpoint.

- › **Inspect logs when:** label generation fails after the upgrade (any carrier) — confirm no regression errors appear in the Request Log tied to the webhook change.
- › **Do not assume upgrade alone is sufficient** for merchants coming from a broken v2.1.8 state — always confirm the delete-and-resync step was completed.
- › **Escalate when:** Webhooks are Active, the recovery flow has been completed, plugin is v2.1.9, and auto-import is still not working — this may indicate a server-level webhook delivery issue or an endpoint authentication problem outside the scope of this fix.
- › **Escalate when:** The Re-sync option does not recreate webhooks as expected on a v2.1.9 site.

Support Escalation Packet

- › Story

ZI-574 - From SL: ZI-574 — Amazon manifest missing FE name, signature, and pickup details [#391952]

Support Guide: ZI-574 — Amazon Manifest Missing FE Name, Signature, and Pickup Details

From SL: ZI-574 — Amazon manifest missing FE name, signature, and pickup details [#391952]

Feature Summary

Prior to this change, Amazon Shipping (carrier C43) manifests generated through PH Multi Carrier Shipping Label App on Shopify were missing three key fields: the Fulfillment Executive (FE) name, a signature placeholder box, and pickup details (date, time window, and location). This left the printed manifest incomplete as a handover document for the pickup driver. The fix adds a dedicated FE block to the Amazon manifest PDF, populated from the scheduled pickup record, and is controlled by a toggle (`manifest.fe.details.enabled.carriers`) that must include "C43" to activate. UPS and all other non-C43 carriers are unaffected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
Feature toggle: <code>manifest.fe.details.enabled.carriers</code> must include "C43"	Confirm with the escalation owner or Ashok that this toggle is enabled on the merchant's store before troubleshooting missing FE block
Toggle gating — when "C43" is absent from the toggle list, the FE block does not render (by design)	If the merchant reports no FE block, check toggle state before assuming a bug
Carrier prerequisite: Amazon Shipping configured as carrier C43 in MCSL	Confirm via hamburger menu > Carriers that Amazon Shipping is present and active

Toggle / config note	What support should confirm
Manifest-eligible orders: At least one Amazon shipment must be in a manifest-ready state	Confirm orders have labels generated and are eligible for manifesting
Customer/test platform: Shopify — mcs1-loggedin354.myshopify.com	Validate on Shopify; do not assume WooCommerce or other platform paths
Release: SL MCSL 382	Confirm the merchant's app version includes this release before troubleshooting
Non-C43 carriers (e.g., UPS): Toggle must NOT include their carrier IDs for isolation	If UPS manifests are affected, confirm toggle value — UPS should be unaffected when toggle = ["C43"] only

Step-by-Step Support Walkthrough

Pre-check: Confirm toggle and carrier setup

1. Open the MCSL app from **Shopify Admin > Apps > PH Multi Carrier Shipping Label App**.
2. Navigate to **hamburger menu > Carriers** and confirm Amazon Shipping is configured and active as carrier C43.
3. Confirm with the escalation owner or Ashok that the toggle `manifest.fe.details.enabled.carriers` includes "C43" for this store. This is a backend/config-level toggle — support cannot verify it from the UI directly; escalate to the dev/config team if uncertain.

Generate and inspect the Amazon manifest (TC-1 — primary verification)

4. Navigate to the **ORDERS tab** in MCSL.
5. Identify one or more Amazon Shipping orders with labels already generated and in a manifest-eligible state.
6. Initiate manifest generation for the Amazon batch. Use the manifest/print flow within the ORDERS area (Print/Download Documents).
7. Download or open the generated manifest PDF.
8. Inspect the PDF for the following FE block elements:
 - › **FE name** — printed in the manifest body
 - › **Signature box** — a clearly labeled, empty/unsigned "Signature" box (not a pre-signed image)
 - › **Pickup details** — date, time window, and pickup location
 - › **Seller details** — seller name present in the manifest header/body
 - › - Request/log fields to verify: FE name field, signature placeholder label, pickup date, pickup time window, pickup location.

Verify multi-batch manifest (TC-2 extension)

9. If the merchant has multiple Amazon batches, generate manifests for more than one batch and confirm the FE block renders correctly across all of them.

Regression: Toggle OFF — confirm FE block is absent (TC-4)

10. If the toggle `manifest.fe.details.enabled.carriers` does NOT include "c43" (or is disabled), generate the Amazon manifest and confirm:

- › No FE block is rendered
- › The manifest layout matches the pre-feature format
- › No crash or rendering error occurs

Regression: Non-Amazon carrier manifest unaffected (TC-5)

11. Navigate to **hamburger menu > Carriers** and confirm UPS is configured.
12. Generate a UPS manifest from the ORDERS tab.
13. Open the UPS manifest PDF and confirm:
- › No FE block appears
 - › The UPS manifest layout is unchanged from prior releases
 - › - Request/log fields to verify: Confirm no FE-related fields are injected into the UPS manifest payload or PDF output.

Regression: Amazon label generation unaffected (TC-7)

14. From the **ORDERS tab**, open an Amazon Shipping order and navigate to **Prepare Shipment / Generate Label**.
15. Generate a single Amazon label (not a manifest).
16. Confirm the label generates successfully, the label PDF is correct, and no FE-block-related errors appear in the request log.
- › - Request/log fields to verify: Check **hamburger menu > Settings > Request Log** (or equivalent log path) for any FE-block-related errors tied to label generation. There should be none.

Post-verification: Cross-check order source data

17. If any order data looks inconsistent, cross-check the order in **Shopify Admin > Orders** to confirm the source order details are correct and the manifest is pulling the right shipment records.

Expected Behaviour

What support should observe	How to confirm
Amazon manifest PDF contains FE name	Open the downloaded manifest PDF — FE name is printed in the FE block section

What support should observe	How to confirm
Manifest PDF contains an empty, labeled "Signature" box	Inspect the PDF — a designated signature box is present and blank (not pre-signed)
Seller name is present in the manifest	Visible in the manifest header or body
Multi-batch manifests all show the FE block	Generate manifests for multiple Amazon batches and inspect each PDF
Toggle OFF → no FE block rendered, no crash	Generate manifest with toggle disabled; layout matches pre-feature format
UPS manifest (non-C43) shows no FE block	Generate UPS manifest; PDF layout unchanged from prior release
Amazon label generation is unaffected by the manifest toggle	Label PDF generates correctly; no FE-related errors in request log
Old/historical Amazon orders manifest correctly	Generate manifest for older Amazon orders; FE block renders when toggle is ON

Merchant-Safe Explanation

Amazon Shipping manifests now include the Fulfillment Executive (FE) name, a signature box for the pickup driver to sign, and the scheduled pickup details (date, time, and location). This makes the manifest a complete handover document that the driver can sign at pickup. This update applies only to Amazon Shipping orders — manifests for other carriers like UPS are not affected. If you are not seeing these details on your manifest, please contact support so we can verify your account configuration.

Common Questions & Troubleshooting

- › **FE name, signature, or pickup details missing from the Amazon manifest:**
 - › First, confirm the feature toggle `manifest.fe.details.enabled.carriers` includes "c43" — this is the most common cause. Escalate to the config/dev team to verify and enable if needed.
 - › Confirm the app version includes MCSL SL 382 or later.
 - › Confirm Amazon Shipping is configured as carrier C43 under hamburger menu > Carriers.
- › **Pickup details are blank on the manifest:**
 - › Collect the manifest PDF and the order/batch details and escalate so the dev/config team can verify the FE block population.
- › **Manifest crashes or fails to render:**
 - › Check the request log (hamburger menu > Settings > Request Log) for errors.
 - › Collect the log output and escalate with the order number and manifest batch details.
- › **UPS or other carrier manifests are showing unexpected FE block content:**
 - › This should not happen when the toggle is set to ["c43"] only. Confirm the toggle value and escalate immediately with the manifest PDF and toggle configuration as a regression.
- › **Amazon label generation is broken after this release:**

› Check the request log for FE-block-related errors during label generation

ZI-575 - From SL: ZI-575 — Canada Post manifest includes unfulfilled orders created same day [#368959]

Support Guide: ZI-575 — Canada Post Selective Manifest Filter

From SL: ZI-575 — Canada Post manifest includes unfulfilled orders created same day [#368959]

Feature Summary

Prior to this change, the Canada Post daily manifest could include same-day orders that had a label generated but had not yet been fulfilled in Shopify — causing those shipments to be transmitted to Canada Post prematurely. This fix introduces a **selective manifest filter** for Canada Post (and Canpar) that restricts manifest entries to orders that are both label-generated **and** fulfilled in Shopify at the time the manifest is run. The filter is toggle-gated and applies only to stores created after a configured date threshold. Unfulfilled orders created on the same day are now correctly excluded from the carrier-side manifest transmission. This affects the Canada Post manifest workflow on Shopify stores running MCSL, with parallel coverage for Canpar.

Toggles & Prerequisites

Toggle / config note	What support should confirm
<code>all.selective.manifest.enabled: true</code>	Must be explicitly set to <code>true</code> in store config for selective filter to activate. Confirm current value before any manifest troubleshooting.
<code>selective.manifest.enabled.after.store.createdAt.greater.than: "2023-01-01"</code>	Store must have been created after 2023-01-01 for this toggle to take effect. Confirm store creation date against this threshold.
Both toggles must be ON together	Selective behavior only activates when both conditions are satisfied. If either is missing or false, legacy manifest behavior applies.
Canada Post carrier account configured	Confirm Canada Post is connected and active under hamburger menu > Carriers.
Canpar carrier account (if applicable)	If the store also uses Canpar, confirm Canpar is configured; selective filter applies to Canpar manifests as well.
Critical migration prerequisite	Before enabling these toggles on any live store, ALL existing label-generated Canada Post orders must be fulfilled in Shopify first. Pre-toggle unfulfilled labeled orders will cause Canada Post errors 9118 and/or 9122 at manifest time.
Test/customer platform	Shopify — validated on <code>mcs1-loggedin354</code> . Confirm store URL and account UUID for any escalation.
Release	SL MCSL 382 iteration backlog. Confirm the store is on a build that includes this release.

Step-by-Step Support Walkthrough

1. Confirm toggle state for the store.

- › Navigate to the store's backend config or request the toggle values from the engineering/escalation team.
- › Verify both `all.selective.manifest.enabled` is `true` and the store's creation date is after 2023-01-01.
- › If either condition is not met, the store is running legacy manifest behavior — do not proceed as if the selective filter is active.

2. Confirm Canada Post (and Canpar, if applicable) is configured.

- › In the MCSL app: **hamburger menu > Carriers**
- › Verify Canada Post appears as an active carrier. If Canpar is also in scope, confirm it is listed and active.

3. Check the fulfillment state of today's orders before the merchant runs the manifest.

- › In **Shopify Admin > Orders**, filter by today's date.
- › Identify which orders have a Canada Post label generated AND are marked as **Fulfilled** in Shopify.
- › Identify any orders that have a label generated but are still **Unfulfilled** — these should be excluded from the manifest under the new behavior.
- › > ☐☐ If the merchant enabled the toggles while labeled-but-unfulfilled orders already existed in the system, flag this immediately as a migration issue before proceeding (see Common Questions & Troubleshooting).

4. Have the merchant (or support, on a test store) generate the Canada Post manifest.

- › In the MCSL app: **ORDERS tab** — locate the manifest generation option for Canada Post.
- › Confirm the merchant is generating the manifest for the correct date (today / same day as label generation).

5. Verify the manifest output against the expected fulfilled count.

- › Open the generated manifest PDF.
- › Count the carrier-side entries and compare against the number of **fulfilled** Canada Post orders for the day.
- › The count of manifest entries must equal the number of fulfilled orders — unfulfilled orders must not appear.
- › - Request/log fields to verify: Canada Post transmission response — confirm no error codes 9118 or 9122 are present. If either appears, treat as a migration issue (pre-toggle unfulfilled labels in the system).

6. Verify the Canada Post carrier-side transmission response in request logs.

- › In the MCSL app: **hamburger menu > Settings > Request Log** (or Automation/Request Log path).
- › Locate the manifest transmission request for Canada Post.
- › - Request/response nodes to verify: Confirm the XML response does **not** contain 9118 or 9122. A clean transmission will return a success response with the correct shipment group count matching fulfilled orders only.

7. Cross-check the ORDERS grid for fulfillment status alignment.

- › In the MCSL app: **ORDERS tab** — filter or review orders for today.

- › Confirm that orders shown as unfulfilled in the ORDERS grid do not appear in the manifest PDF.
- › Confirm that fulfilled orders' tracking numbers in the manifest match the label log entries for today.

8. If Canpar is configured on the same store, repeat steps 4–7 for a Canpar manifest.

- › The selective filter applies to Canpar manifests using the same toggle logic.
- › Verify that only fulfilled Canpar orders appear in the Canpar manifest output.

9. Verify toggle-OFF / legacy behavior is unaffected (regression check).

- › On a store where `all.selective.manifest.enabled` is `false` OR the store was created before 2023-01-01, confirm the manifest behavior is unchanged from the prior release.
- › No regression to the legacy manifest workflow should be observed.

Expected Behaviour

What support should observe	How to confirm
Manifest PDF contains entries only for fulfilled Canada Post orders	Count manifest entries; compare to Shopify Admin > Orders filtered by Fulfilled + today's date
Unfulfilled same-day orders are excluded from the manifest	Unfulfilled orders do not appear in the manifest PDF; their tracking numbers are absent
Manifest entry count matches fulfilled label count exactly	Manifest PDF entry count = number of fulfilled CP orders with labels generated today
Canada Post transmission succeeds with no error codes	Request log shows success response; no 9118 or 9122 in XML response
Voided shipments are excluded from the manifest	Voided labels do not appear in the manifest PDF; manifest count = fulfilled-and-not-voided shipments only
If no fulfilled orders exist for the day, a clear UI message is shown	UI displays a "no fulfilled orders for manifest" type message; no empty PDF is generated
Toggle OFF stores retain legacy manifest behavior	Manifest on toggle-OFF stores matches prior release behavior; no regression observed
Canpar manifests follow the same selective filter logic	Canpar manifest PDF contains only fulfilled Canpar orders when toggles are ON
Same-day fulfilled orders are included (not excluded by date logic)	Orders created and fulfilled on the same day as manifest generation appear correctly in the manifest
Pre-toggle unfulfilled labeled orders cause Canada Post errors 9118 / 9122	Request log XML response contains 9118 (stale group id) or 9122 (all groups empty/excluded) when migration step was skipped

Merchant-Safe Explanation

When you generate your Canada Post daily manifest, the app now checks whether each order has actually been fulfilled in Shopify before including it in the transmission to Canada Post. Orders that have a shipping label generated but have not yet been marked as fulfilled will be left out of the manifest automatically — so you won't accidentally transmit shipments that aren't ready to go. This means your manifest will always reflect only the orders you've confirmed are packed and ready, even if other orders

were created or labeled on the same day. **Important:** if you are enabling this feature for the first time, please make sure all of your existing labeled Canada Post orders are marked as fulfilled in Shopify before turning on the new settings — otherwise the manifest may return an error from Canada Post.

Common Questions & Troubleshooting

- › **First thing to check:** Confirm both toggles are active (`all.selective.manifest.enabled: true` and store creation date > 2023-01-01). If either is missing, the store is on legacy behavior and the selective filter is not running.
- › **Merchant reports manifest count is lower than expected:** Verify in Shopify Admin > Orders that the "missing" orders are actually marked as Fulfilled. If they are still Unfulfilled, the filter is working correctly — the merchant needs to fulfill those orders before running the manifest.
- › **Canada Post error 9118 (Group id was not found for transmission):** This indicates pre-toggle labeled orders with stale group IDs are in the system. The merchant enabled the toggles before fulfilling all existing labeled orders. Escalate — a one-time backfill or manual resolution of the stale group IDs is required.
- › **Canada Post error 9122 (All groups in the transmit request were empty or all shipments were excluded; there was nothing to transmit):** All shipments were filtered out — either no orders are fulfilled for the day, or the migration step was skipped and all labeled orders are pre-toggle unfulfilled. Check fulfillment state in Shopify Admin > Orders and confirm whether the migration prerequisite was followed.
- › **When to inspect request logs:** Any time a Canada Post manifest transmission returns an error, pull the request log from **hamburger menu > Settings > Request Log** and look for the XML response body. Check for 9118 or `9

ZI-576 - From SL: ZI-576 — FedEx Saturday Delivery shown as surcharge instead of separate checkout [#391628]

Support Guide: ZI-576 — FedEx Saturday Delivery Separate Checkout Rate

From SL: ZI-576 — FedEx Saturday Delivery shown as surcharge instead of separate checkout [#391628]

Feature Summary

Previously, FedEx Saturday Delivery was surfaced as a surcharge on top of an existing service rate rather than as a distinct, selectable option at checkout. This change corrects that behaviour: when Saturday Delivery is applicable for a shipment, the Saturday-eligible variant of each qualifying FedEx service (e.g., **FedEx Priority Overnight Saturday Delivery**) now appears as a separate line item alongside the standard service (e.g., **FedEx Priority Overnight**) in the checkout rate list. The feature is gated behind a per-account toggle (`fedex.rest.saturday.delivery.rates.enabled`) and applies to FedEx REST API rate calls only. UPS and DHL rates are unaffected. The fix ensures merchants and their customers can make an explicit, informed choice between standard and Saturday delivery, and that the service selected at checkout is the service used when the label is generated.

Toggles & Prerequisites

Toggle / config note	What support should confirm
<code>accountuuid.fedex.rest.saturday.delivery.rates.enabled: true</code>	Confirm this toggle is enabled at the account UUID level before any Saturday Delivery troubleshooting. If absent or false, Saturday rates will not appear regardless of ship date.
FedEx REST API integration (not legacy SOAP)	Confirm the store's FedEx carrier connection is using the REST API. This toggle and behaviour apply to the REST implementation only.
Rate Summary display	Confirm Rate Summary is active and returning FedEx rates at checkout. Saturday Delivery lines will only appear within the rate summary output.
Ship date eligibility (Thursday or Friday checkout)	Saturday Delivery is only returned by FedEx when the ship date qualifies. Confirm the order's ship date falls on a Thursday or Friday, or that the destination/service combination is Saturday-eligible per FedEx rules.
FedEx carrier account configured in MCSL	Confirm a valid FedEx account is connected under hamburger menu > Carriers. Without an active FedEx account, no FedEx rates — Saturday or otherwise — will be returned.
One Rate and/or Account Rates enabled	Feature works with Account Rates only, One Rate only, or both. When both are enabled, only the lowest of the two Saturday rates is shown. Confirm which rate type(s) are active for the store.
Customer/test platform	Shopify (keee-qa-store confirmed as QA store). Validate on Shopify Admin + MCSL app.
MCSL release	SL MCSL 382 iteration backlog. Confirm the store is on a release that includes this change before troubleshooting.

Step-by-Step Support Walkthrough

1. Confirm the toggle is enabled for the account.

- › Navigate to the account's backend configuration and verify `accountuuid.fedex.rest.saturday.delivery.rates.enabled` is set to `true` for the merchant's account UUID.
- › If the toggle is missing or false, Saturday Delivery rates will not appear. This is the first check for any "Saturday rate not showing" report.

2. Confirm FedEx is configured correctly in MCSL.

- › In the MCSL app (Shopify Admin > Apps > PH Multi Carrier Shipping Label App), go to **hamburger menu > Carriers**.
- › Verify a FedEx account is connected and active.
- › Confirm whether Account Rates, One Rate, or both are enabled for the FedEx carrier configuration.

3. Confirm the ship date qualifies for Saturday Delivery.

- › Saturday Delivery eligibility is determined by FedEx based on ship date (typically Thursday or Friday) and destination.
- › Ask the merchant to confirm the order's intended ship date and destination ZIP/postal code.

› If the ship date does not qualify, FedEx will not return a Saturday Delivery rate — this is expected behaviour, not a bug.

4. Reproduce the checkout rate display on the Shopify storefront.

- › Place a test order on the Shopify storefront (or use the merchant's reported order) with a Saturday-eligible ship date and a FedEx-served destination.
- › At checkout, verify the rate list shows **both** the standard service (e.g., *FedEx Priority Overnight*) **and** the Saturday variant (e.g., *FedEx Priority Overnight Saturday Delivery*) as two separate, independently selectable options.
- › Verify the Saturday Delivery option is priced higher than the base service option.
- › Verify no Saturday Delivery option appears when the ship date is not eligible.

5. Inspect the rate request and response logs.

- › In the MCSL app, go to **hamburger menu > Settings** (or **Request Log** if available in the navigation).
- › Pull the rate log for the relevant order or checkout session.
- › - Request/response nodes to verify: confirm a parallel Saturday Delivery rate request is fired alongside the standard rate request; confirm the Saturday Delivery response contains a separate rate entry distinct from the base service rate; confirm the Saturday surcharge is merged into all available services that support Saturday delivery (i.e., the surcharge is included within each Saturday-eligible service's rate).
- › If the toggle is OFF or the ship date is ineligible, confirm **no** Saturday parallel request is present in the log.

6. Verify duplicate Saturday rate suppression.

- › When both Account Rates and One Rate are enabled and both return a Saturday Delivery rate, confirm only **one** Saturday Delivery line appears at checkout — the one with the lowest rate.
- › - Request nodes to verify: confirm only the lowest-priced Saturday Delivery rate entry is surfaced in the rate summary output; no duplicate Saturday lines should appear.

7. Verify the selected Saturday service carries through to label generation.

- › In the MCSL app, go to the **ORDERS** tab, open the relevant order, and navigate to **Prepare Shipment / Generate Label**.
- › Confirm the Order Summary displays the Saturday Delivery service that was selected at checkout.
- › Generate the label and confirm the label reflects the Saturday-eligible service.
- › - Request/log fields to verify: confirm the FedEx label request includes the Saturday Delivery flag/special service indicator; confirm the generated label PDF or tracking number corresponds to a Saturday Delivery service.

8. Verify non-Saturday FedEx services are unaffected.

- › On the same store, confirm that standard FedEx services (e.g., FedEx 2Day, standard Priority Overnight) continue to appear and are priced correctly.
- › Confirm enabling Saturday Delivery has not altered rates for non-Saturday services.

9. Verify UPS and DHL rates are unaffected (regression check).

- › If UPS and/or DHL are configured alongside FedEx on the store, confirm their rates at checkout are unchanged and no Saturday Delivery lines appear under those carriers.

10. Verify return shipments (if applicable).

- › If the merchant uses return labels, confirm that FedEx Saturday Delivery is also listed correctly for return shipments and that label printing for return Saturday Delivery works as expected.
- › Reference QA log: `SaturdayDelivery_Return.zip` (attached to Trello card).

11. Verify ancillary features are unaffected (regression).

- › Confirm SLGP, QuickShip, Edit Package, Signature, and Insurance continue to function correctly alongside Saturday Delivery.
- › Confirm international FedEx shipments and FedEx Freight shipments are unaffected.
- › Reference QA sanity logs: `FedExInternatioanl.zip`, `FedExFreight.zip` (attached to Trello card).

Expected Behaviour

What support should observe	How to confirm
Saturday-eligible checkout shows two separate FedEx service lines: the standard service and the Saturday Delivery variant	Shopify storefront checkout rate selector shows both options as independently selectable rows
Saturday Delivery option is priced higher than the base service	Rate values visible in the checkout rate list; Saturday line > base line price
Non-eligible shipments show no Saturday Delivery option	Place a test order with a non-Thursday/Friday ship date or ineligible destination; Saturday line absent from checkout
Toggle OFF suppresses all Saturday Delivery rates even for eligible shipments	Disable <code>fedex.rest.saturday.delivery.rates.enabled</code> ; no Saturday line appears at checkout; no parallel Saturday request in rate log
When both Account Rates and One Rate are active, only one Saturday Delivery line appears (lowest rate)	Checkout shows a single Saturday Delivery entry; rate log confirms deduplication
Service selected at checkout matches service used at label generation	MCSL ORDERS tab > order > Prepare Shipment shows the Saturday service; generated label reflects Saturday Delivery
FedEx label request includes Saturday Delivery special service flag	Rate/label request log node confirms Saturday Delivery indicator is present in the FedEx API request
Standard FedEx services (2Day, standard Priority Overnight, etc.) are priced and displayed as before	Checkout rate list and rate log show no change to non-Saturday service rates
UPS and DHL rates are unchanged	Checkout rate list shows UPS/DHL rates identical to pre-feature baseline
Return shipments list Saturday Delivery correctly and labels print successfully	MCSL return label flow; label PDF reflects Saturday Delivery service
International and Freight FedEx shipments unaffected	Checkout and label generation for international/freight orders behave as before

What support should observe	How to confirm
Service names displayed exactly as: <i>FedEx Priority Overnight</i> and <i>FedEx Priority Overnight Saturday Delivery</i>	Checkout rate selector label text matches these exact strings

Merchant-Safe Explanation

When Saturday Delivery is available for your shipment, you and your customers will now see it listed as a separate shipping option at checkout — for example, *FedEx Priority Overnight Saturday Delivery* — alongside the standard *FedEx Priority Overnight* option. Previously, the Saturday Delivery cost was being added as a surcharge rather than shown as its own choice, which made it unclear

ZI-577 - From SL: ZI-577 — Love of India: batch UX, retry state corruption, print mismatch, carri [#390510]

Support Guide: ZI-577 — Batch UX, Retry State Corruption, Print Mismatch, Carrier Consistency

From SL: ZI-577 — Love of India: batch UX, retry state corruption, print mismatch, carri [#390510]

Feature Summary

This card addresses four distinct batch label generation defects surfaced by the Love of India merchant (a high-volume Shopify store) and fixed across two PRs (#3104, #3064), deployed as part of MCSL-381P. The fixes cover: (1) large batches timing out because the HTTP response waited for full batch completion instead of returning immediately on queue; (2) retry after partial failure corrupting batch state by re-generating or voiding already-successful labels; (3) the Print Documents action producing a mismatched PDF (wrong page count relative to successful labels); and (4) carrier selection drifting mid-batch so some labels silently used a different carrier than the one selected. All four issues affect the ORDERS/batch label flow on Shopify with FedEx and UPS as the relevant carriers.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	No gating — fix is live for all stores on MCSL-381P and later
Release prerequisite	Store must be on MCSL-381P or later; confirm app version in Shopify Admin > Apps
Carrier account prerequisite — FedEx	Active FedEx account configured under hamburger menu > Carriers; credentials valid
Carrier account prerequisite — UPS	Active UPS account configured under hamburger menu > Carriers; credentials valid
Customer / test platform	Shopify (Love of India store context; validate on Shopify)
High-volume batch prerequisite	Batch behaviour is most observable with 50+ orders; Love of India-style config uses 200-order batches
Packaging configuration	Confirm packaging rules are set under hamburger menu > Settings / Packaging before batch testing

Step-by-Step Support Walkthrough

A — Batch UX: Large batch queues without HTTP timeout

1. In **Shopify Admin > Apps > PH Multi Carrier Shipping Label App**, navigate to the **ORDERS** tab.

2. Select 50 or more orders (ideally 200 for a Love of India-scale reproduction). Click **Generate Labels (Batch)**.

3. Observe the HTTP response timing in the browser network panel or MCSL request log (**hamburger menu > Settings / Shipping Rates / Automation or Request Log**).

› - Request/log fields to verify: batch initiation response time, returned batch_id, HTTP status code (expect 2xx within a few seconds – not a 5xx or timeout)

4. Confirm the UI immediately transitions to a **queued** or **processing** state and displays a batch ID — it must not hang waiting for all labels to complete before returning.

5. Watch the batch progress panel for incremental counter updates (e.g. 25/200 → 50/200 → ... → 200/200). The final state must show **total processed** and **total failed** counts.

B — Retry state corruption: Retry targets only failed orders

6. From the **ORDERS** tab, open a completed batch that has a mix of successful and failed orders (e.g. 45 successful, 5 failed due to address errors).

7. Click **Retry** on the batch.

8. Confirm in the batch detail UI that **exactly the 5 failed orders** are re-attempted.

› - Request/log fields to verify: order IDs submitted in the retry request – must match only the failed order IDs, not the full batch

9. Confirm the 45 already-successful labels are **not voided or re-generated**. Cross-check tracking numbers in **Shopify Admin > Orders** — no duplicate tracking numbers should appear for the previously successful orders.

10. For a network-blip scenario: if a batch was interrupted mid-run, re-open the batch and click **Resume / Retry**. Confirm only not-yet-processed orders are re-attempted and that done/failed/pending counts are accurate with no overcounting.

› - Request/log fields to verify: batch state fields (done_count, failed_count, pending_count) before and after retry; confirm done_count does not reset

C — Print mismatch: PDF page count matches successful labels exactly

11. From the **ORDERS** tab, open a completed batch (e.g. 50 successful labels, or 45 successful + 5 failed).

12. Click **Print Documents** for the batch.

13. Count the pages in the resulting PDF.

› For a 50-success / 0-failed batch: PDF must contain **exactly 50 pages**.

› For a 45-success / 5-failed batch: PDF must contain **exactly 45 pages** — no placeholder pages for failed orders.

14. Cross-check each page's tracking number against the batch detail UI tracking number list. Every page must correspond to a tracking number in the batch; no extras or missing pages are acceptable.

› - Request/log fields to verify: label document IDs included in the print request – count must equal successful label count only

D — Carrier consistency: Batch carrier does not drift mid-batch

15. From the **ORDERS** tab, create a batch with a specific carrier selected (e.g. **FedEx**).

16. After the batch completes, open the batch detail view and review the per-order carrier column.

17. Confirm every label in the batch shows **FedEx** (or whichever carrier was selected). No order should silently show a different carrier (e.g. UPS).

› - Request/log fields to verify: carrier field per label record in the batch detail; cross-check against the carrier selected at batch creation

18. Repeat for a UPS batch to confirm symmetry.

E — Single-order regression check

19. From the **ORDERS** tab, open a single non-batch order and click **Generate Label** (via **Prepare Shipment / Generate Label** on the order detail).

20. Confirm the label generates normally with no batch/queue-related side effects (no unexpected batch ID assigned, no state errors, label and tracking number appear correctly in the Order Summary and Label Summary).

21. Cross-check the generated label in **Shopify Admin > Orders** to confirm tracking is synced correctly.

Expected Behaviour

What support should observe	How to confirm
Batch initiation returns a batch ID within a few seconds for 200-order batches	Browser network panel or MCSL request log: HTTP 2xx response with batch_id, response time < timeout threshold
Batch UI shows queued → processing → completed state transitions with incremental counters	Watch the batch progress panel in the ORDERS tab during a live batch run
Retry submits only failed order IDs, not the full batch	Request log: retry payload contains only failed order IDs; batch detail UI shows only those orders re-attempted
Already-successful labels are not voided or duplicated after retry	Shopify Admin > Orders: tracking numbers for previously successful orders remain unchanged; no duplicate tracking numbers
Batch state counters (done/failed/pending) are accurate after retry or resume	Batch detail UI: done_count reflects pre-retry successes + retry successes; no reset or overcounting

What support should observe	How to confirm
Print Documents PDF page count equals the number of successful labels exactly	PDF page count matches successful label count; failed orders produce no placeholder pages
Every PDF page tracking number matches a tracking number in the batch detail	Cross-check PDF pages against batch detail tracking number list — 1:1 match
All labels in a FedEx batch show FedEx; all labels in a UPS batch show UPS	Batch detail per-order carrier column; no carrier drift visible
Single-order label generation is unaffected by batch changes	Order Summary + Label Summary show normal label; no batch ID or queue artefacts

Merchant-Safe Explanation

We've made several improvements to how batch label generation works for high-volume shipments. Large batches now start processing immediately without causing a timeout — you'll see a batch ID and progress updates right away rather than waiting for everything to finish at once. If some labels in a batch fail, clicking Retry will now only re-attempt those specific failed orders, so your successful labels stay exactly as they are with no risk of duplicates. The Print Documents feature now produces a PDF with exactly the right number of pages — one per successful label, with no extras or missing pages. Finally, the carrier you choose when creating a batch (such as FedEx or UPS) will now be applied consistently to every label in that batch.

Common Questions & Troubleshooting

- › **Batch still timing out after the fix?** First confirm the store is on MCSL-381P or later (Shopify Admin > Apps > PH Multi Carrier Shipping Label App — check the app version). If the version is correct, capture the network response time and HTTP status from the browser network panel and escalate with that evidence.
- › **Retry is re-generating successful labels?** Check the retry request payload in the MCSL request log (hamburger menu > Settings / Shipping Rates / Automation or Request Log) — confirm whether the payload contains only failed order IDs or the full batch. If the full batch is being submitted, this is a regression and should be escalated immediately with the log.
- › **PDF page count is wrong after the fix?** Count the successful labels in the batch detail UI and compare to the PDF page count. If they do not match, capture the batch ID, the batch detail UI screenshot, and the PDF and escalate.
- › **Carrier drift observed?** Open the batch detail and screenshot the per-order carrier column. Note the carrier selected at batch creation. If any order shows a different carrier, escalate with the batch ID and screenshot — do not ask the merchant to void and regenerate until escalation confirms the root cause.
- › **Single-order flow broken?** If a merchant reports that individual label generation stopped working after this release, check whether a batch ID is being incorrectly assigned to the single order in the Order Summary. Capture the order number and Label Summary screenshot and escalate.

ZI-578 - From SL: ZI-578 — Add YunExpress as supported carrier for China-origin shipments [#388157]

Support Guide: ZI-578 — YunExpress Carrier Integration

From SL: ZI-578 — Add YunExpress as supported carrier for China-origin shipments [#388157]

Feature Summary

This card adds YunExpress as a fully supported carrier within the PH Multi Carrier Shipping Label (MCSL) app on Shopify. The integration covers carrier registration, credential management, rate fetching, label generation, tracking, void/cancel, manifest, QuickShip, Label Batch, and return flows. YunExpress is scoped exclusively to **China-origin cross-border shipments** (e.g., CN → US, CN → EU); domestic shipments are not supported by the carrier. The integration spans three backend repositories (carrier-registration-api, ship-rate-track proxy, storepep-react, and storepep-tracking-api). UPS and DHL are listed as regression carriers — support should confirm neither is affected by this addition. QA was conducted on a Shopify store using mock rates in the QA environment; live credential validation should be confirmed on a production-equivalent store.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggle detected	Feature should be available to all eligible Shopify merchants after release; no gating flag to check
Release: SL MCSL 382	Confirm the merchant's store is on MCSL app version 382 or later before troubleshooting
Shopify platform only (per card/QA scope)	Validate on Shopify; do not assume WooCommerce/BigCommerce/Magento/PrestaShop parity without a separate card
YunExpress carrier account required	Merchant must have a valid YunExpress account with App ID, Source Key, and Application Key
China-origin store or ship-from address required	YunExpress will not return rates for non-CN origin addresses; confirm store/ship-from origin before troubleshooting missing rates
Pickup not supported by YunExpress	Do not attempt to configure or troubleshoot YunExpress pickup — carrier does not support it
Special services not supported by YunExpress	No special service options are expected in the UI for this carrier
Domestic shipments not supported	YunExpress is cross-border only; CN → CN orders will not produce rates
Return flow — fixed and verified	The return flow was failing during QA but has since been fixed and verified . Returns work as expected; no special handling required
Custom product orders — known issue at QA time	Orders with custom products were marked failing (❌) during QA; confirm current status from live store/card context
Mandatory fields for return	The return mandatory-fields gap seen during QA has been resolved as part of the return fix; returns are verified working

Toggle / config note	What support should confirm
UPS / DHL regression	Confirm UPS and DHL continue to function normally on non-CN-origin orders after YunExpress is added

Step-by-Step Support Walkthrough

Part 1 — Carrier Registration & Credential Setup

1. In the Shopify Admin, open **Apps > PH Multi Carrier Shipping Label App**.
2. Navigate to **Hamburger menu > Carriers**.
3. Click **Add Carrier** and confirm **YunExpress** appears in the supported carriers list with its logo.
 - › If YunExpress is not listed, confirm the store is on MCSL app version 382 or later.
4. Select **YunExpress** and enter the merchant's credentials:
 - › **App ID**
 - › **Source Key**
 - › **Application Key**
5. Click **Test Connection**.
 - › A success message should appear for valid credentials.
 - › An error/validation message should appear for invalid credentials — the carrier should not be saveable in that state.
6. Click **Save** and confirm YunExpress appears as **Active** in the Carriers list.

Part 2 — Rate Fetching

7. Go to the **ORDERS** tab in MCSL.
8. Open a China-origin order (ship-from CN, ship-to US/EU or other supported cross-border destination).
9. Click **Prepare Shipment** and then **Get Rates**.
10. Confirm YunExpress rate option(s) appear in the **Rate Summary**.
 - › If no rates appear, check the ship-from address — non-CN origins will not return YunExpress rates.
 - › Rates were verified using mock rates in QA; on a live store, confirm the merchant's YunExpress account is active and credentialed correctly.
 - › - Request/log fields to verify: carrier name in rate response, origin country code (should be CN), rate amount, service name returned by YunExpress.
11. For a non-CN origin order, attempt to fetch YunExpress rates and confirm a clear unsupported/error message is shown — no rate should be generated.

Part 3 — Label Generation

12. In the **ORDERS** tab, open the China-origin order, select a YunExpress rate, and click **Generate Label**.

13. Confirm:

- › A YunExpress label PDF is generated and downloadable.
- › A YunExpress tracking number is displayed on the **Order Summary**.
- › - Request/response nodes to verify: label request payload (origin country = CN), tracking number in response, label URL/PDF reference in response.

Part 4 — Packaging Types & Multi-Package

14. In the **ORDERS** tab, open the order and navigate to **Prepare Shipment**.

15. Confirm supported YunExpress packaging types are available in the packaging selector.

- › Also check **Hamburger menu > Settings > Packaging** for any store-level packaging configuration.

16. Test with a **single package** and a **multiple package** order — both should produce valid labels.

17. Use **Edit Package** (both old and new package edit flows) and confirm changes persist correctly before label generation.

Part 5 — SLGP, Manifest, QuickShip, Label Batch

18. SLGP (Ship Label Generate Print): Confirm the SLGP flow produces a YunExpress label without errors.

19. Manifest: After generating labels, confirm a manifest can be created for YunExpress shipments.

20. QuickShip: Confirm QuickShip generates a YunExpress label directly without the standard rate-selection flow.

21. Label Batch: Confirm multiple YunExpress orders can be processed in a batch label run from the **ORDERS** tab.

- › - Request/log fields to verify: batch request carrier assignment, individual label responses per order in the batch.

Part 6 — Void / Cancel Shipment

22. In the **ORDERS** tab, open an order with a generated YunExpress label.

23. Click **Void / Cancel Label** on the **Order Summary**.

24. Confirm:

- › The label is cancelled at the YunExpress carrier side.
- › The order returns to a pre-label state in MCSL.
- › - Request/response nodes to verify: cancel/void request sent to YunExpress, success/confirmation response, order status reset in MCSL.

Part 7 — Return

25. Attempt to generate a return label for a YunExpress shipment from the **ORDERS** tab > **Order Summary**.

26. Confirm the return label generates successfully. The return flow was failing during QA but has since been **fixed and verified** — returns now work as expected.

› - Request/response nodes to verify: return request sent to YunExpress, success/confirmation response, and return label generated in MCSL.

Part 8 — Tracking

27. In the **ORDERS** tab, open an order with a generated YunExpress label.

28. Click the YunExpress tracking number on the **Order Summary**.

29. Confirm the YunExpress tracking site opens in a new tab with the correct tracking ID populated in the URL.

› - Request/log fields to verify: tracking number format, tracking URL domain (should be YunExpress tracking portal), tracking status returned.

Part 9 — Discounts

30. Confirm that any configured shipping discounts apply correctly to YunExpress rates in the **Rate Summary**.

› Navigate to **Hamburger menu > Settings > Shipping Rates / Automation** to check discount rules if rates appear without expected discounts.

Part 10 — Zero-Dimension / Zero-Weight Products

31. Test with a product configured with **0 dimensions and 0 weight** in **Hamburger menu > Products**.

32. Confirm the system defaults the weight to **0.01 kg** and still returns a rate (verified in QA).

› - Request/log fields to verify: weight value sent in the rate request — should be 0.01 kg when product weight is 0.

Part 11 — UPS & DHL Regression Check

33. With YunExpress active, open a **non-CN-origin order** in the **ORDERS** tab.

34. Fetch rates and confirm **UPS** and **DHL** rates appear as expected.

35. Generate a UPS or DHL label and confirm it completes without errors.

36. Confirm no cross-carrier interference from the YunExpress addition.

Part 12 — Pickup (Expected Non-Support)

37. Confirm the **PICKUP** tab does not offer YunExpress as a pickup carrier.

38. If a merchant reports missing YunExpress pickup, inform them the carrier does not support pickup — this is expected behaviour.

Expected Behaviour

What support should observe	How to confirm
YunEx	

ZI-579 - From SL: ZI-579 — FedEx REST SLGP not displaying signature option on generated labels [#389953]

Support Guide: ZI-579 — FedEx REST SLGP Signature Option on Generated Labels

From SL: ZI-579 — FedEx REST SLGP not displaying signature option on generated labels [#389953]

Feature Summary

Prior to this fix, merchants using FedEx SmartPost / Ground Economy (SLGP) services via the FedEx REST integration in MCSL on Shopify could not see or select a **Signature Required** option on the Single Label Generation page. This meant SLGP shipments were going out without signature handling even when the merchant intended to require it. The fix surfaces the **Signature Required** toggle and its sub-types (Direct Signature, Indirect Signature, Adult Signature, None) on the Single Label page for SLGP orders, ensures the selected sub-type is sent correctly in the FedEx REST label request payload, and allows Favorite Presets to persist the signature selection. UPS signature flow and other FedEx services (Ground, Express, Priority) are expected to be unaffected.

Toggles & Prerequisites

Toggle / config note	What support should confirm
accountuuid.show.and.edit. signature.on.quick.summary .details.section.enabled	Confirm this toggle is set to true for the merchant's account UUID in the feature flag / config store before testing or troubleshooting
673a1996-1a9d-4cc1-9c59-52 7b3f9c5f20.show.and.edit.s ignature.on.quick.summary. details.section.enabled	This is the known test/reference UUID toggle; confirm the merchant's own account UUID variant is also enabled if the above generic toggle is not sufficient
Release prerequisite	Store must be on SL MCSL 382 or later; confirm app version before investigating missing UI
Carrier account prerequisite — FedEx REST	Merchant must have a FedEx REST account configured in MCSL (not FedEx SOAP/legacy); SLGP (SmartPost / Ground Economy) must be an enabled service on that account
SLGP-eligible order	An order must be eligible for FedEx SmartPost / Ground Economy service; confirm service appears in the rate/service dropdown on the Single Label page
Customer / test platform	Shopify — all navigation paths below are Shopify-specific

Toggle / config note	What support should confirm
UPS regression scope	UPS carrier account should also be configured if verifying the UPS regression test case (TC-8)

Step-by-Step Support Walkthrough

1. Confirm the toggle is active for the merchant's account.

Navigate to the feature flag / config store and verify both toggle keys listed above are set to `true` for the merchant's account UUID. If either toggle is missing or `false`, the Signature Required UI will not appear regardless of service selection. Do not proceed to UI verification until this is confirmed.

2. Confirm the app version on the merchant's Shopify store.

In **Shopify Admin > Apps > PH Multi Carrier Shipping Label App**, check the displayed app version or build reference. The fix ships in **SL MCSL 382**. If the store is on an earlier version, escalate for a version update before continuing.

3. Open an SLGP-eligible order in MCSL.

In **Shopify Admin > Orders**, locate an order that is eligible for FedEx SmartPost / Ground Economy. Open it, then switch to the **MCSL ORDERS** tab and open the order record there.

4. Navigate to the Single Label Generation page.

From the MCSL order record, click **Prepare Shipment / Generate Label** to open the Single Label Generation page. In the service selector, choose a **FedEx SLGP (SmartPost / Ground Economy)** service.

5. Verify the Signature Required toggle is visible.

On the Single Label Generation page, confirm a **Signature Required** toggle is present in the shipment options area. It should be **unchecked / OFF by default**.

› *If the toggle is absent:* re-check step 1 (feature flag) and step 2 (app version) before escalating.

6. Enable the Signature Required toggle and verify sub-types.

Turn the **Signature Required** toggle ON. Confirm a sub-type dropdown appears immediately below or adjacent to the toggle. The dropdown must list at minimum:

- › Direct Signature
- › Indirect Signature
- › Adult Signature
- › None

7. Generate a label with Direct Signature selected and inspect the request log.

Select **Direct Signature** from the sub-type dropdown, then click **Generate Label**. After generation, navigate to **hamburger menu > Settings** (or the **Request Log** section within MCSL) and locate the most recent FedEx label request for this order.

› - Request nodes to verify: `specialServicesRequested` → `specialServiceTypes` should contain the Direct Signature service code; `signatureOptionDetail` → `optionType` should reflect **DIRECT** (or the FedEx REST equivalent code). Confirm the label PDF also indicates Direct Signature.

8. Repeat for Adult Signature.

Return to the Single Label Generation page for the same or a comparable SLGP order. Select **Adult Signature** and generate the label. Check the request log again.

- › - Request nodes to verify: `specialServicesRequested` → `specialServiceTypes` and `signatureOptionDetail` → `optionType` should reflect ADULT (or the FedEx REST equivalent). Confirm the label PDF reflects Adult Signature.

9. Verify Signature Required OFF sends no signature service (regression check).

Open another SLGP order, ensure **Signature Required** is OFF, and generate the label. Inspect the request log.

- › - Request nodes to verify: Confirm `specialServicesRequested` does **not** contain any signature service code and `signatureOptionDetail` is absent or empty. The label PDF should show no signature requirement.

10. Verify Favorite Preset persists Indirect Signature.

If the merchant uses Favorite Presets: configure a preset with FedEx SLGP + **Indirect Signature** selected, save it, then apply it to a new SLGP order on the Single Label page. Confirm the preset auto-fills Indirect Signature. Generate the label and check the request log.

- › - Request nodes to verify: `signatureOptionDetail` → `optionType` should reflect INDIRECT (or the FedEx REST equivalent).

11. Verify other FedEx services are unaffected (regression).

Open an order using **FedEx Ground, Express, or Priority** (non-SLGP). Confirm the Signature Required toggle still appears and functions as it did before this fix. Generate a label with a signature sub-type selected and inspect the request log to confirm the correct sub-type is sent.

- › - Request nodes to verify: Same `specialServicesRequested` and `signatureOptionDetail` nodes as above; confirm values match the selected sub-type.

12. Verify UPS signature flow is unaffected (regression).

Open a UPS-configured order in the **MCSL ORDERS** tab, generate a label with UPS signature options enabled, and inspect the request log.

- › - Request/log fields to verify: Confirm UPS signature fields are present and correct in the UPS request payload; confirm no FedEx-specific signature nodes are appearing in the UPS request. The UPS label PDF should reflect the expected UPS signature handling.

Expected Behaviour

What support should observe	How to confirm
Signature Required toggle visible on Single Label page for SLGP	Toggle is present and unchecked by default when a FedEx SmartPost / Ground Economy service is selected on the Single Label Generation page
Sub-type dropdown appears when toggle is turned ON	Dropdown lists Direct Signature, Indirect Signature, Adult Signature, and None
Direct Signature sent in FedEx REST payload	Request log: <code>specialServicesRequested</code> → <code>specialServiceTypes</code> contains Direct Signature code; <code>signatureOptionDetail</code> → <code>optionType</code> = DIRECT (or FedEx REST equivalent)

What support should observe	How to confirm
Adult Signature sent in FedEx REST payload	Request log: signatureOptionDetail → optionType = ADULT (or FedEx REST equivalent); label PDF confirms Adult Signature
Indirect Signature sent in FedEx REST payload	Request log: signatureOptionDetail → optionType = INDIRECT (or FedEx REST equivalent); label PDF confirms Indirect Signature
No signature node in payload when toggle is OFF	Request log: specialServicesRequested contains no signature service code; signatureOptionDetail absent or empty
Favorite Preset round-trips signature selection	Applying a saved preset auto-fills the saved signature sub-type on the Single Label page; request log confirms the sub-type is sent on label generation
Other FedEx services (Ground / Express / Priority) unaffected	Signature Required toggle and sub-types continue to work as before on non-SLGP FedEx services; request log confirms correct sub-type
UPS signature flow unaffected	UPS label request log contains correct UPS signature fields; no FedEx signature nodes present in UPS requests

Merchant-Safe Explanation

If you use FedEx SmartPost or Ground Economy (also called SLGP) to ship orders, you can now select a **Signature Required** option directly on the label generation page — just like you can for other FedEx services. Once you turn on the Signature Required option, you can choose the type of signature you need: Direct, Indirect, Adult, or the FedEx service default. Your choice will be applied to the label and sent to FedEx automatically. You can also save your preferred signature setting in a Favorite Preset so it applies every time you use that preset.

Common Questions & Troubleshooting

- › Signature Required toggle is not

ZI-580 - From SL: ZI-580 — Add PostNL service code 6942 (Boxable Track and Trace) to available se [#379796]

Support Guide: ZI-580 — PostNL Service Code 6942 (Boxable Track & Trace)

From SL: ZI-580 — Add PostNL service code 6942 (Boxable Track and Trace) to available services [#379796]

Feature Summary

This change adds the following **PostNL** service codes to the available PostNL service list in MCSL (PR #3109, postnlConstants.js / serviceCodes.js). Carrier scope: **PostNL only**. Validated platform: **Magento**.

Service code	Service name
6942	Boxable Track & Trace (contract)
6350	Packet Track & Trace

Service code	Service name
6405	Packet Untracked
6440	Boxable Untracked
6906	Packet Track & Trace insured
6972	Boxable Track & Trace

What Support Should Confirm

- › The store is on **MCSL 382 or later** — the codes appear only after the release is deployed.
- › **PostNL** is configured and active under hamburger menu > Carriers.
- › A rate is returned and a label generates for an EU Packet-eligible order using the selected code; the request log shows the correct service code in the payload.
- › The printed label may display "Packet Tracked Standard" — this is PostNL's own label text for these service codes and is expected behaviour.

Merchant-Safe Explanation

The PostNL service codes listed above are now selectable in your PluginHive Multi Carrier Shipping Label app, alongside your existing PostNL services. Select a code, fetch rates, and generate labels for eligible shipments as usual. The printed label may show "Packet Tracked Standard" — this is normal and comes directly from PostNL.

Migrate FedEx REST DG, Alcohol, and Battery shipment support from C2 to C39 and Implemented ORM_D dg shipment - Migrate FedEx REST DG, Alcohol, and Battery shipment support from C2 to C39 and Implemented ORM_D dg shipment

Support Guide: MCSL 382 — FedEx REST DG, Alcohol & Battery Migration (C2 → C39) + ORM-D

Migrate FedEx REST DG, Alcohol, and Battery shipment support from C2 to C39 and Implemented ORM_D DG shipment

Feature Summary

This change migrates FedEx dangerous goods (DG), alcohol, and battery shipment support from the legacy C2 carrier type to the C39 (FedEx REST) carrier type within MCSL. Prior to this work, merchants using FedEx REST for special commodity shipments were still relying on C2-era data structures, which caused mismatches in payload construction and missing fields in the REST request. The migration covers all three commodity flows — DG fields (regulation, technical name, accessibility, DG option, ORM-D), alcohol fields (recipient type, licensee type), and battery fields (lithium-ion and lithium-metal) — and introduces ORM-D support natively in FedEx REST (C39). A backend migration script was run to copy existing C2 commodity data to C39 records. This affects FedEx REST label generation, rate requests, and product-level DG configuration on Shopify stores using MCSL.

Toggles & Prerequisites

Toggle / config note	What support should confirm
No feature toggles detected	Feature is live for all FedEx REST (C39) stores in SL MCSL 382 scope; no per-store flag to enable
FedEx REST (C39) carrier must be configured	Confirm the store has an active C39 carrier under hamburger menu > Carriers — not just C2
Migration script must have run for the store	Confirm C39 DG/alcohol/battery data is populated on the product; if blank, the migration may not have covered this account (see audit note: 882 accounts missing C39 could not be auto-copied)
Products must have DG/alcohol/battery configured at product level	Confirm under hamburger menu > Products — open the product and check the Special Commodity / DG section
Alcohol: destination state must permit direct alcohol shipping	Confirm the ship-to state is not restricted; state-gating is enforced at label generation
ORM-D: previously SOAP-only	For existing customers who had ORM-D in C2/SOAP, confirm the field is now visible and populated in C39 UI after migration; if blank, escalate
cargoAircraftOnly flag	Known issue: this node is passed in the label/shipment request but was not included in the rate request at time of QA — confirm from live store/card context whether this has been resolved in 382
Hazardous Materials (existing customers)	For stores with Hazardous Materials already enabled pre-migration, confirm that additional mandatory DG fields (Proper Shipping Name, Container Type, Packing Instructions, Quantity Units, Quantity Type) are now visible in the UI; if missing, label generation for HAZARDOUS_MATERIALS may fail
Customer/test platform	Shopify — validate in Shopify Admin > Apps > PH Multi Carrier Shipping Label App

Step-by-Step Support Walkthrough

Part A — Confirm Carrier and Product Setup

1. Confirm FedEx REST (C39) carrier is active.

Navigate to hamburger menu > Carriers in the MCSL app. Confirm a FedEx REST carrier (C39 type) is present and connected. If only C2 is present, the migration cannot apply and the store needs C39 configured before proceeding.

2. Confirm DG/alcohol/battery data migrated to the product.

Navigate to hamburger menu > Products. Open the relevant product. Check the Special Commodity / Dangerous Goods section. Confirm that C39-side fields are populated:

- › For DG: Regulation, DG Technical Name, Accessibility, DG Option, and (for Hazardous Materials) Proper Shipping Name, Container Type, Packing Instructions, Quantity Units, Quantity Type.
- › For Alcohol: Recipient Type (e.g., licensee).
- › For Battery: Battery type (lithium-ion or lithium-metal) and associated packing fields.
- › For ORM-D: Confirm the ORM-D option is visible and not blank.

> If any of these fields are blank for a store that previously had C2 DG data, check whether the account was in the "missing C39 / cannot auto-copy" group (882 accounts identified in the migration audit). Escalate if confirmed.

3. Confirm the order contains the DG/alcohol/battery product.

In Shopify Admin > Orders, open the relevant order and confirm the line item matches the product configured in step 2.

Part B — Rate Request Verification

4. Trigger a rate fetch for the DG/alcohol/battery order.

In the MCSL app, go to the ORDERS tab, open the order, and proceed to Prepare Shipment. Confirm rates are returned for FedEx REST services.

> If rates are not returned for a DG-enabled store, this matches a known issue identified in QA. Inspect the rate request log immediately.

5. Inspect the rate request log for DG/alcohol/battery nodes.

Navigate to hamburger menu > Settings > Request Log (or the equivalent log path for this store). Filter for the FedEx REST rate request for the order.

Request nodes to verify: `dangerousGoodsDetail` (regulation, technical name, accessibility, DG option), `specialServicesRequested` for alcohol, battery type fields (lithium-ion / lithium-metal), `ormD` flag for ORM-D variants.

- › **Known gap to check:** Confirm whether `cargoAircraftOnly` is present in the rate request. Per QA, this node was missing from rate requests at test time — DG with Cargo Aircraft Only was a failing case. If still absent, note for escalation.

Part C — Label Generation Verification

6. Generate the FedEx REST label for the order.

In the ORDERS tab, open the order, go to Prepare Shipment / Generate Label, select the FedEx REST service, and generate the label.

7. Inspect the label request log for DG/alcohol/battery nodes.

In hamburger menu > Settings > Request Log, filter for the FedEx REST label/shipment request.

Request/response nodes to verify:

- › DG: `dangerousGoodsDetail` containing regulation, DG technical name, accessibility, DG option, container type, packing instructions, quantity units, quantity type (for Hazardous Materials).
- › Alcohol: alcohol special-service node, adult signature flag (if applicable), recipient/licensee type.
- › Battery: lithium-ion or lithium-metal battery fields, packing instruction reference.
- › ORM-D: `ormD` or equivalent ORM-D designation node in the C39 payload.
- › `cargoAircraftOnly`: confirm present in label request (QA confirmed this passes in label request).

- › Confirm FedEx returns a success response (no 4xx errors). A 4xx for Hazardous Materials likely indicates missing mandatory DG fields in the payload.

8. Verify the generated label and documents.

Download the label PDF and any accompanying documents (commercial invoice, dangerous goods declaration). Confirm:

- › DG declarations are present for DG shipments.
- › Alcohol indicator is shown for alcohol shipments.
- › Lithium-ion or lithium-metal indicator and packing instruction reference appear for battery shipments.

9. Confirm no fallback to C2 is occurring.

In the request log, confirm the carrier ID in the label request corresponds to C39 (FedEx REST), not C2. If C2 is still being used for DG label generation post-migration, this is a regression — escalate immediately.

Part D — ORM-D Specific Verification

10. Verify ORM-D is available and functional in C39.

In hamburger menu > Products, open a product previously configured with ORM-D under C2/SOAP. Confirm the ORM-D field is visible and populated in the C39 UI. If blank, the migration did not carry over this value for the account.

Request nodes to verify: ORM-D designation in the C39 label request payload. QA confirmed ORM-D Inaccessible Limited and ORM-D Accessible Limited passed in both rate and label requests.

Expected Behaviour

What support should observe	How to confirm
FedEx REST (C39) label generates successfully for DG, alcohol, and battery products	Label PDF downloaded without error; FedEx returns 2xx response in request log
dangerousGoodsDetail node present in C39 label request with regulation, technical name, accessibility, DG option	Request log — label request payload
For Hazardous Materials: Proper Shipping Name, Container Type, Packing Instructions, Quantity Units, Quantity Type visible in UI and present in payload	hamburger menu > Products UI + request log
Alcohol fields (recipient type, licensee type, adult signature) present in C39 label request and shown on label PDF	Request log + label PDF

What support should observe	How to confirm
Lithium-ion battery fields present in C39 label request; lithium-ion indicator on label/documents	Request log + label/document PDFs
Lithium-metal battery fields present in C39 label request; lithium-metal indicator on label/documents	Request log + label/document PDFs
ORM-D field visible in C39 product UI and present in label request payload	hamburger menu > Products + request log
cargoAircraftOnly node present in label/shipment request	Request log — label request payload
cargoAircraftOnly node present in rate request	Request log — rate request payload (<i>confirm from live store — known gap at QA time</i>)
Rates returned for DG-enabled FedEx REST orders	ORDERS tab >

Iteration 2: FedEx REST DG Close Manifest – Show Confirmation Modal Indicating Manifest Will Be Generated for All FedEx Ground Closed Label Orders - Iteration 2: FedEx REST DG Close Manifest – Show Confirmation Modal Indicating Manifest Will Be Generated for All FedEx Ground Closed Label Orders

Support Guide: MCSL 382 – FedEx REST DG Close Manifest Confirmation Modal

Iteration 2: FedEx REST DG Close Manifest – Show Confirmation Modal Indicating Manifest Will Be Generated for All FedEx Ground Closed Label Orders

Feature Summary

Prior to this change, when a merchant triggered **Generate Manifest** (via Mark as Fulfill) for a FedEx Ground Dangerous Goods order, no warning was shown about the scope of the manifest operation. FedEx's manifest generation is not limited to the selected order — it covers **all eligible FedEx Ground orders with closed labels that have not yet been manifested for the current day**. This iteration adds a **confirmation modal** that surfaces this scope to the merchant before the manifest is generated, requiring explicit confirmation or cancellation. The modal is gated: it appears **only when the Dangerous Goods (DG) toggle is enabled and the carrier service is FedEx Ground**. All other services and DG-off configurations are unaffected. This change is part of the FedEx REST DG Close Manifest enhancement and is scoped to the Shopify platform in this release.

Toggles & Prerequisites

Toggle / Config Note	What Support Should Confirm
Dangerous Goods (DG) toggle — feature is gated; modal only appears when DG is enabled	Confirm DG toggle is ON in hamburger menu > Carriers > FedEx carrier settings before testing or troubleshooting
Carrier service must be FedEx Ground	Confirm the order's selected service is FedEx Ground specifically; modal will not appear for FedEx Priority, Express, or other FedEx services

Toggle / Config Note	What Support Should Confirm
FedEx REST carrier account	Confirm the store is using FedEx REST (not legacy SOAP); confirm from hamburger menu > Carriers
Closed labels prerequisite	Manifest generation only includes orders with closed labels for the current day; confirm labels have been generated and closed before triggering manifest
Release prerequisite	Feature is part of SL MCSL 382 iteration backlog; confirm the store is on MCSL 382 or later
Platform	Shopify; validate on Shopify Admin > Apps > PH Multi Carrier Shipping Label App
Fulfillment entry points in scope	Modal is triggered from: Order Summary (Mark as Fulfill), Orders Grid Bulk Action (Mark as Fulfill), Label Batch page (Mark as Fulfill), Today's Labels, Label Created Tab, New Tab, Cancel and Retry, Custom Product Fulfillment, Partially Externally Fulfilled Order flows
Fulfillment entry points NOT in scope (no modal)	SLGP Fulfillment, Quick Ship Fulfillment, Manual Fulfillment — confirm these do not show the modal as expected

Step-by-Step Support Walkthrough

1. Confirm the store is on MCSL 382 or later.

- › Check the app version or release tag visible in the MCSL app footer or confirm from the internal release tracker.

2. Confirm FedEx REST is configured and DG is enabled.

- › In the MCSL app on Shopify Admin > Apps > PH Multi Carrier Shipping Label App, go to **hamburger menu > Carriers**.
- › Open the FedEx carrier account and confirm:
 - › The account is using **FedEx REST** (not legacy).
 - › The **Dangerous Goods toggle is ON**.
 - › If DG is OFF, the modal will not appear — this is expected gated behaviour (see TC-7).

3. Confirm a qualifying order exists.

- › Navigate to the **ORDERS** tab in MCSL.
- › Identify an order that:
 - › Uses **FedEx Ground** as the carrier service.
 - › Has a **closed label already generated** for the current day.
 - › Has **not yet been manifested today**.
- › If no such order exists, the modal may not trigger or the empty-state message should appear after confirmation.

4. Trigger Mark as Fulfill from Order Summary (primary verification path — TC-1).

- › In the **ORDERS** tab, open the qualifying FedEx Ground DG order.
- › Click **Mark as Fulfill** from the Order Summary view.
- › **Expected:** A confirmation modal appears immediately before any manifest is generated.
- › Verify the modal contains:

- › A clear message stating that manifest generation will apply to **all eligible FedEx Ground closed-label orders for the current day**, not only the selected order.
- › A **Confirm** button to proceed.
- › A **Cancel** button to abort.

5. Verify modal content is accurate (TC-7 modal text check).

- › Read the modal body text carefully.
- › Confirm it explicitly states the scope is **all eligible FedEx Ground orders** (day-wide), not just the selected order(s).

6. Test the Confirm path (TC-4).

- › Click **Confirm** in the modal.
- › **Expected:** Manifest generation is triggered for all eligible FedEx Ground closed-label orders for the current day — including orders that were not part of the original selection.
- › Verify a success toast or summary is shown, ideally with a count of manifested orders.
- › Check that a manifest PDF is generated and downloadable.
- › - **Request/log fields to verify:** Confirm in the request log (hamburger menu > Settings > Request Log or equivalent) that the manifest request was sent to FedEx REST and that the response includes confirmation for all eligible orders, not only the selected order's shipment ID.

7. Test the Cancel path (TC-5).

- › Trigger Mark as Fulfill again on a qualifying order to re-open the modal.
- › Click **Cancel**.
- › **Expected:** The modal closes. No manifest is generated. The order remains in its pre-action state. The merchant stays on the same page.
- › Confirm no manifest log entry or PDF was created for this cancel action.

8. Verify the modal appears from all in-scope fulfillment entry points.

- › **Orders Grid Bulk Action (TC-2):** In the **ORDERS tab**, select multiple FedEx Ground DG orders and use **Bulk Action > Mark as Fulfill**. Confirm the same modal appears.
- › **Label Batch page (TC-3):** Navigate to the Label Batch page (accessible from the ORDERS tab or hamburger menu as applicable). Open a batch containing FedEx Ground DG orders and click **Mark as Fulfill**. Confirm the modal appears.
- › **Today's Labels, Label Created Tab, New Tab, Cancel and Retry, Custom Product Fulfillment, Partially Externally Fulfilled Order:** Trigger Mark as Fulfill from each of these entry points and confirm the modal appears consistently.

9. Verify the modal does NOT appear when DG is disabled (TC-7 regression).

- › Go to **hamburger menu > Carriers**, open the FedEx account, and **disable the DG toggle**.
- › Return to the **ORDERS tab**, open a FedEx Ground order, and trigger Mark as Fulfill.
- › **Expected:** No confirmation modal appears. The flow proceeds as it did before this feature.
- › Re-enable DG after this check.

10. Verify the modal does NOT appear for non-FedEx Ground services (TC-8).

- › Identify or create an order using a non-Ground FedEx service (e.g., FedEx Priority Overnight) with DG enabled.
- › Trigger Mark as Fulfill.
- › **Expected:** No confirmation modal appears. The existing fulfillment flow continues unchanged.

11. Verify entry points confirmed as no-modal (regression guard).

- › Trigger fulfillment via **SLGP Fulfillment**, **Quick Ship**, and **Manual Fulfillment** paths.
- › **Expected:** No modal appears on any of these paths, consistent with QA sign-off notes.

12. Verify only current-day closed-label orders are included (TC-13).

- › After confirming and generating a manifest, review the manifest PDF or success summary.
- › Confirm that orders from previous days are **not** included in the manifest.
- › - **Request/log fields to verify:** Check the manifest request payload in the request log for the date scope parameter; it should reflect the current day only.

13. Verify duplicate manifest prevention (TC-15).

- › After a successful manifest generation, trigger Mark as Fulfill again on the same order(s).
- › **Expected:** Already-manifested orders are not re-manifested. Confirm from the manifest PDF count or success summary that duplicates are excluded.

Expected Behaviour

What Support Should Observe	How to Confirm
Confirmation modal appears when Mark as Fulfill is triggered for a FedEx Ground DG order with a closed label	Trigger from Order Summary, Bulk Action, or Label Batch; modal must appear before any manifest action
Modal text clearly states manifest scope is ALL eligible FedEx Ground closed-label orders for the current day, not only the selected order	Read modal body text; confirm wording matches this scope description
Confirm button triggers manifest generation for all eligible current-day FedEx Ground closed-label orders	Check manifest PDF and success toast/count after clicking Confirm
Cancel button closes the modal without generating any manifest	No manifest PDF created; no manifest entry in request log; order state unchanged
Merchant remains on the same page after clicking Cancel	UI does not navigate away; order/batch page remains active
Modal does NOT appear when DG toggle is OFF	Disable DG in hamburger menu > Carriers > FedEx; trigger Mark as Fulfill; no modal shown
Modal does NOT appear for non-FedEx Ground services (e.g., FedEx Priority) even with DG ON	Use a non-Ground FedEx service order; trigger Mark as Fulfill; no modal shown
Modal does NOT appear from SLGP Fulfillment, Quick Ship, or Manual Fulfillment paths	Trigger from these paths; confirm no modal; existing flow proceeds

What Support Should Observe	How to Confirm
Only current-day closed-label orders are included in manifest scope	Review manifest PDF order count; cross-